

Final Design Review

Submitted to The University of Manchester for the degree of
Master of Science in Robotics
in the Faculty of Science and Engineering

Year of submission

2026

Student ID

Team 9 (14341814, 11103012, 14255884, 14215307, 14316539, 10826766)

School of Engineering

Contents

Contents	2
1 Introduction	5
1.1 Problem Statement and Objectives	5
1.1.1 Problem Statement	5
1.1.2 Background	6
1.1.3 Target users	6
1.1.4 Objectives	7
1.1.5 Constraints and Requirements	7
1.1.6 Success criteria	7
1.2 Addressing PDR Feedback	8
1.2.1 Mechanical Design	8
1.2.2 Software Diagram	8
1.2.3 Analysis	8
1.2.4 Project Plan	9
1.3 Modifications since PDR	9
1.3.1 Mechanical Design	9
1.3.2 Electrical Design	9
1.3.3 Manipulator Design	10
1.3.4 Software Design	10
1.3.5 Analysis	10
1.3.6 Project Plan	11
2 Sustainability Checklist	11
2.1 Materials [7]	11
2.2 Software	12
2.3 Energy	13
2.4 Waste*	13
2.5 Emissions	14
2.6 Communications	14
2.7 Modularity	15
2.8 Location/Placement	16
2.9 Maintenance	16
2.10 Repurposing	16
3 Cyber Security Considerations	17

4 Updated Preliminary Design Review	18
4.1 System Design	18
4.1.1 System Components and Justification	18
4.1.2 Summary	20
4.2 Mechanical Design	21
4.2.1 Design Overview	21
4.2.2 Design Considerations	22
4.2.3 Component Placement Considerations	24
4.2.4 Material Choice and Fabrications	25
4.2.5 Strength	25
4.2.6 Stability	26
4.3 Electrical Design	28
4.3.1 Power Connection Diagram	28
4.3.2 Power Budget Calculations	29
4.3.3 Proposed Solution	30
4.4 Software Design	31
4.4.1 Software Diagram	31
4.4.2 State machine	31
4.4.3 Scan Filtering	34
4.4.4 Frontier Exploration	35
4.4.5 Approaching Object	37
4.4.6 Manipulator	38
4.4.7 Object Perception	38
4.4.8 Team GitHub Repository	39
4.5 Analysis: Sub-Team Progress and Implementation Strategy	39
4.5.1 Navigation and Mapping	39
4.5.2 Manipulator	42
4.5.3 Achieved Requirements & their Analysis	42
4.5.4 Object Perception	46
4.5.5 Power Requirements	47
4.6 Project Plan	47
References	50
5 Appendix	51
5.1 Appendix A: Design Requirements	51
5.1.1 Problem Statement	51

5.1.2	Concept of Operations (ConOps)	51
5.1.3	Functional and Performance Requirements	53
5.1.4	Requirements Verification Matrix	55
5.2	Appendix B: Workplace Charter	58
5.2.1	Purpose	58
5.2.2	Principles and Commitments	59
5.2.3	Team Rules	59
5.2.4	Daily Activity Conflict Avoidance	60
5.2.5	Conflict Resolution Guidelines	60
5.2.6	Conflict Resolution Flowchart	60

Word count:

1 Introduction

The objective of this project is to design and develop a fully autonomous robot capable of navigating a rectangular search area with randomly placed obstacles to collect 3 coloured cylinders and return them to matching-coloured bins at the rover's starting point within a 20-minute mission time.

1.1 Problem Statement and Objectives

1.1.1 Problem Statement

In recent years, artificial intelligence and machine learning have been excelling in many industrial applications such as computer vision, decision making, control and navigation. With each of these sub-branches being gradually mastered by AI models, the main challenges to come is the combination of all these functions to form a single intelligent system as a whole.

From a research perspective, it is always much simpler to focus the development on a single aspect of application and maximize its performance. For instance, one can invest in years of efforts and money to develop the world's best CNN model which has billions of parameters and takes months to train. Or one can use the most complex algorithm to design a robotic control system that enables a humanoid robot to perform the most complex motions existing. But in practice, modern, consumer-level intelligent products often requires multiple AI systems to streamline and work together in the most efficient possible way. And this is often a challenging task as many newly developed, fresh-out-of-the-labs AI systems are either too large or complex to be integrated in consumer products, or cannot be easily interfaced with other systems without spending weeks solving compatibility issues.

This often leads to many problems among researcher, engineers and consumers. For instance, any potential new researchers maybe be less inclined to work in system integration as are often attracted to research in safer directions such as sole vision or control. For engineers who are developing a product to complete a certain task, they would often be confused to select the best available AI model or spend too much time attempting to combine multiple intelligent systems together. As consumers, the result is a more expensive product and a more complicated user experience.

These issues are amplified further for fully autonomous robots. These types of robots often requires multiple sensors, actuators, and tools to achieve their expected functionality and performance. They also requires minimal to no human intervention, meaning all system components must remain fully autonomous without sacrificing performance or accuracy. This puts even more press in sub-system integration and sensor fusion.

In our project, we have developed an autonomous mobile robotic solution which features a 4 wheel rover capable of navigation in any open space whilst performing path planning, object recognition and object manipulation tasks. Our project aims to address the challenges mentioned above while providing an easy to use, cheap to manufacture robotic product.

1.1.2 Background

Autonomous mobile robots have existed since the first intelligent robot. But despite their simplified imagery as "a smart car", their complexity cannot be underestimated and continues to expand as our expectation and demand for an AI systems increases over the years.

The most basic functionality for any mobile robots is the ability to navigate in open environments. Robotic navigation typically involves mapping, localization, path planning and trajectory tracking. The importance of a decent navigation system is manifested by the fact that it fundamentally limits any additional functionality the robot may perform. And many research have been solely focusing on navigation alone, with most effort being directed to localization/mapping and path planning [1].

Vision and sensing are also both essential for these class of robots to perform many day-to-day tasks. There have been many machine learning methods for robots computer vision, from traditional, algorithm based methods such as BoVW, SVM or clustering, to CNN based, deep learning models [2], [3], [4]. Apart from image based sensing, other exteroceptive sensors that are widely used in autonomous robots are Lidars, which uses laser technologies and is often used to generate a detailed map of the robot's environment. Many research has existed in developing faster and more efficient Lidar systems for robots [5], [6].

Lastly, autonomous robots are often equipped with an intelligent tool that is flexible and adaptive to complete tasks such as handling, grasping and placing. A common choice is a robotic manipulator. These class of robots typically have high degree of freedom (DoF) and is able to reach complex spaces and can interact with a great variety of objects.

1.1.3 Target users

Our mobile robot offers a compact design and is suitable for open space exploration and salvation tasks involving multiple small objects. It is especially useful in environment that could be hazardous to human workers. The robot uses economic materials and a modular design which allows for any consumers, researchers or hobbieists to work on.

In summary, our robot is suitable for both industrial applications as well as laboratory researching.

1.1.4 Objectives

The aim of the project is to develop multiple intelligent systems and then integrate them together as a whole to achieve complex tasks. The project should aim to achieve as much functionality as possible without sacrificing performance or increasing the price of the robot by too much.

We also aim to optimize the power of the robot so that it can operate for longer, allowing it to complete more tasks. It should also be environmentally and economically sustainable according to the UN's 17 sustainability goals. Moreover, it should be user friendly and easy to use/develop to any consumers or new researchers.

1.1.5 Constraints and Requirements

As a typical autonomous mobile robot, our robot is designed to:

1. Navigate fully autonomously in any known or unknown environments.
2. Being able to recognize obstacles and objects while generating a map of the environment.
3. Perform object manipulation including picking, placing, re-orienting using a manipulator.
4. Based on ros2 and feature custom ros nodes and algorithms which can perform decision making.

We are given a leorover, an Intel NUC and a Mycobot 280 Pi manipulator as our only available main hardware for this project. We are also constrained to use an Intel realsense depth camera and an RP-A2 Lidar for sensing and navigation.

1.1.6 Success criteria

Individual systems are first tested to function as intended before integrated and test the whole system. This is summarized in the following table.

Sub-system	Success criteria	Yes	No
Lidar	Maps the environment	Success	fail
Camera	Recognize objects and compute their locations	Success	Fail
Navigation	Moves rover in open space without hitting obstacles/get stuck	Success	Fail
Manipulator	Handles objects well	Success	Fail
Whole robot	Completes common tasks in open environment	Success	Fail

1.2 Addressing PDR Feedback

1.2.1 Mechanical Design

In the PDR, the design philosophy seemed to be lacking sufficient detail, especially regarding the choice of acrylic as the material for the base platform. In this final design review, this feedback has been acted upon by providing additional details regarding decision choices including the choice of using acrylic for the base and upper platforms. This can be found in Section 4.2.4.

Previously, manufacturability was only discussed implicitly, with limited emphasis on how design choices influenced fabrication and assembly. This has been addressed by introducing explicit discussion of manufacturing methods and constraints, including the rationale for selecting laser-cut acrylic for planar components and 3D printed PLA for complex geometries. Consideration has also been given to factors such as ease of fabrication, repeatability, prototyping speed and assembly. Ensuring that design decisions are clearly justified from a manufacturability perspective. This can be seen in Section 4.2.4.

Initially only CAD model images were provided and without CAD drawings this limits their usefulness for manufacturing and replication. This has been addressed by including fully dimensioned drawings for the payload sled, upper platform, base platform, and onboard storage bin in Sections 4.2.1 and 4.2.2. This ensures that components can be accurately manufactured and assembled.

1.2.2 Software Diagram

Previously, the software block diagram had a poor resolution, making it difficult to read all the details. This happened because the block diagram contains a high density of information and there was a loss in quality when attempting to include all the information about nodes, topics, services and components in one diagram. In this document, the block diagram has been updated to reflect the nodes we are using currently, rather than the ones we planned to use while completing the PDR. The quality of the new diagram has also been improved as seen in Section 4.4.1.

In the PDR, the state machine was only briefly mentioned and no additional diagrams or explanations were provided to share how our state machine operates. To improve this, the current document contains a state machine flow diagram with detailed explanation of what occurs in each state, this is shown in Section 4.4.2. Section 4.4 also contains RQT graphs to further visualise how the nodes operate.

1.2.3 Analysis

Some functional or performance requirements had no testing method associated with them. We have noticed that the Requirements Verification Matrix (RVM) and their associated testing methods were missing. Only paraphrased testing procedures were outlined. In Section 4.5, we have included a table with detailed analysis of how each parts of our design satisfies the requirements. This ensures all relevant sections have their testing method and an explanation of how the requirements were satisfied.

Some language could be more specific to ensure the reader understands all the relevant terminology and details. Throughout this document, we have explained the technical details and some implicitly mentioned procedures more clearly, as well as added references for the paragraphs so that the readers can refer back to the corresponding sections for more information.

Testing procedures were not described in enough detail. It is true that tests were only paraphrased but not described in detail. Moreover, the relationships and correspondence between each test and

their associated requirements are not clarified. For each design solution that satisfies the design requirements, there is an accompanying testing section outlining the entire test procedure. This ensures any further testing can be carried out by any new user to verify the function and performance of the rover product.

Fulfilled requirements were not explicitly summarized or presented to the reader. Due to the lack of RVM in the the corresponding analysis sections, the extent of fulfilment of the requirements described in this section are ambiguous. Throughout Section 4.5, we have included summary tables with detailed analysis of all satisfied requirements as well as how each requirements are fulfilled. This clearly summarizes all requirements that were satisfied. The reader can easily refer back to the previous sections or to the Appendix using the requirements ID to perform checks.

Success criteria were not explicitly described. We have describes possible testing methods to verify the fulfilment of the requirements, but none of them have a clear description of what counts as a successful test. Each tables in Section 4.5 now has an accompanying success criteria clearly outlined and defined, with their associated verification methods. This ensures the user to immediately know whether a test has successfully satisfied the requirement or not.

1.2.4 Project Plan

Previously, milestones were not included within the Gantt chart, but they were presented separately in a second figure. Since, milestones are essential for documenting team progress. We have now added milestones to the Gantt chart. This clearly shows all stages of the project have been completed on time, indicating a successful team work overall.

1.3 Modifications since PDR

1.3.1 Mechanical Design

Since the PDR, the depth camera has been moved from the upper platform to the base platform. This had to be done because the camera's view of the cylinder was extremely limited when the rover approaches it. This meant that the distance and coordinate data from the camera became unreliable and noisy. It was discovered that moving the depth camera to be on the base platform would put it level with the cylinder, significantly improving perception.

Due to the addition of the two Anker power banks, a 3D printed power bank caddy was also added to the payload sled, this is attached to the rear of the base platform using two bolts.

As new mounting holes had to be created to facilitate the depth camera and the power bank caddy on the base platform, the acrylic base platform had to be manufactured again through laser cutting. This opportunity was also used to add access slots in the base platform so that excess cables could be guided through the 10mm spacing between the rover's chassis and the base platform.

1.3.2 Electrical Design

In our old design in PDR, in the power connection diagram, the Intel NUC mini pc will communicate wirelessly with the Mycobot manipulator. In our final design we modified it so that the Intel NUC mini pc is communicating via ethernet with the Mycobot manipulator. This is because wireless communication requires setting up before the operation. Ethernet connection is permanent once the IP address from both devices are linked. The ethernet cable also offers a larger bandwidth and provides are more stable connection. This allows for more information to be sent to the manipulator and reduces communication errors/failures.

Similarly, in the PDR, the intel NUC was designed to communication wirelessly with the rover's Raspberry Pi 4 process. We decided to use a usb-to-ethernet cable to connect the intel NUC to the rover in our final design. This is due to the wireless connections being too slow, having a negative impact on the navigation functionality and performance, since navigation requires a high bandwidth. Hence using an ethernet connection was more suitable.

In the power budget calculation of our PDR, we planned to buy an additional power bank to power the secondary system (See Power Connection Diagram). In our final design, we will be using the power bank provided by the Laboratory. Following the discussion with the Laboratory teaching staff, we have requested the access of an additional power banks with no extra cost. The power banks also features a 3 channel output which provides more power demand and can include backup in the case of accidental failure.

1.3.3 Manipulator Design

In PDR, we have planned to use external path planning library such as Moveit2 to perform trajectory tracking and obstacle avoidance. In our final design, we have decided to code a custom path planning algorithm. This is due to the fact that the tasks of picking and placing cylinders does not require complex planning. The environment does not contain complex obstacles. Running a complete simulation using Moveit2 in Rviz then perform a sim-to-real transfer is unnecessary since we are using Mycobot's hardware-only motion planners. Pre-defined rules and simple interpolation is sufficient for the tasks given.

In PDR, the Intel NUC mini pc shall receive information from depth camera and performs the pick-and-place algorithm and send control signal to the Mycobot manipulator via wireless communication to command the manipulator's hardware IK solver. In our final design, the Intel NUC mini pc shall only receive object information from the depth camera. It shall now use a publisher node and ethernet connection to send object information to the manipulator, who should receive it via a subscriber node with the same topic. This is because using Intel NUC to perform pick-and-place tasks solely requires setting up port links between the mini pc and the Manipulator's internal hardware. This is difficult as only the manipulator's Raspberry Pi usb ports are exposed and communication through usb only can cause port id conflicts. Moreover, the pick-and-place algorithm should only be executed once per new object data. Using a publisher-subscriber framework allows for the pick-and-place function to be triggered according to some conditions. The ethernet cable also allows for permanent connection between the Intel NUC and the manipulator through IP address handshaking.

1.3.4 Software Design

In our PDR, we used software diagram to show all front-end and back-end integration. Included descriptions of the each algorithm including approaching, grasping and sorting. As a modification, added state machine to represent how every software programs are integrated. Added RQT graphs. Added more detailed description of the software components for every sub-system of the rover. This offers:

- A clearer overview of the software system.
- A more detailed and professional description of the functionality of each software program and how they are related to the rover's hardware and system design.

1.3.5 Analysis

In our PDR, we summarized the testing procedure and design as paragraph for each section to perform requirements analysis. In addition to the summary, we added tables of detailed analysis includ-

ing design solutions, requirements fulfilments, testing methods and success criteria for each section in our final design. This ensures:

- All requirements outlined in the Requirements Verification Matrix are shown to be satisfied.
- All requirements have a relevant testing method associated with it to make sure it can be properly verified.
- All tests have a clearly defined success criteria.

1.3.6 Project Plan

The Gantt chart in our PDR summarizes each team member's assigned tasks as well as outlining the project's timeline. In addition to the summary, we added milestones to each sub-team to monitor progress in our final design. This ensures each check point of the project is completed in time and allows the progress of the team to be validated.

2 Sustainability Checklist

2.1 Materials [7]

Chassis Main Body, (Materials: *Aluminum alloy*)

The Aluminum metal is 100 % recyclable [8], has a high strength-to-weight ratio and low manufacturing/recycling cost and lowers weight and transportation energy. Moreover, recycling does not degrade its value. Aluminum offers supreme re-usability, malleability and cost-effectiveness. It's been widely used in many industrial fields, including mobile robotics.

Chassis Motor Holder, (Materials: *3D-printing PLA*)

PLA is 100 % recyclable [9],[10] and bio-degradable (breaks down within twelve weeks) and cheap-/Can be purchased in bulk. Overall, PLA is a versatile class of 3D-printing plastic that is durable, cost-effective and easily reusable.

Storage Bin, (Materials: *3D-printing PLA*)

Description: See **Chassis Motor Holder**

Onboard-component Board, Materials: *Acrylic*

Acrylic material [11] is highly recyclable, light weight and has lower transportation energy. It's durable with long lifespan and is cost-effective and easy to maintain. Similar to PLA, acrylic is another class of versatile plastic that is easily recyclable and easy to cut/manufacture. Can use CNC-cut aluminum board or 3D printing materials.

Tyre, (Materials: *Rubber*)

Rubber [12],[13] is renewable and cheap/easily replaceable Can use omniwheels made of recyclable, bio-degradable 3D printed polymer such as PLA. Bio-rubber is also a more sustainable alternative as it's bio-degradable and reduces carbon dioxide footprint.

Mycobot Manipulator, (Materials: *Polymer, aluminium, steel/metal*)

Description: Design constraint, cannot be changed, N/A

Manipulator Gripper [14], (Materials: ABS)

ABS [15] is another class of highly sustainable plastic and is highly recyclable. Can use PLA or PETG 3D printing materials. Can increase production energy and cost due to the use of 3D-printer but the materials can be more recyclable or bio-degradable.

Buhler Motors, (Materials: Aluminium, Carbon)

The Buhler Motor company [16] has claimed that 50 % of their production uses renewable energy. The company has been using greentech to lower production energy, waste and increasing the percentage of recyclable and renewable materials. Overall, the Buhler brushed motors are among the most sustainable products of brushed motors and was certified to be environmentally and ecologically friendly. To maximize sustainability, brushless motors can be used for their improved energy efficiency, longer life and lower noise.

Samsung 18650 INR Battery, (Materials: Lithium)

Lithium has high energy efficiency/density. Li-ion batteries offers low carbon footprint than traditional fuels and is recyclable. [17] Can replace with LFP (Lithium Iron Phosphate) batteries which are less toxic, have a longer lifespan and lower carbon footprint, and are safer to use. For renewable options, can consider using sodium ion batteries.

Wiring & Connectors, (Materials: Aluminum, Copper, PVC)

These materials are: 1. Recyclable. 2. Have high production efficiency. 3. Non-toxic and light weight. 4. Cheap and cost-effective. The wiring materials provides good durability and are sustainable in terms of recycling and production energy cost.

2.2 Software

CNN model training Training and evaluating deep learning models requires intensive GPU operations that increases GPU/CPU power usage. Modern CPUs typically operating at 65-150W while GPUs operates at even higher wattage from 30-450W. Training with large dataset and model is energy expensive and increases CO2 footprint. The model is designed to only complete the minimal amount of tasks to complete the project. This ensures our models are light and can be trained on CPU which is less energy intensive. We have selected our dataset carefully and by using data argumentation algorithms to ensure our models generalizes faster in terms of recognizing the target objects.

CNN model inference The trained model is downloaded and stores in the Intel NUC mini pc. It receives image data from the depth camera and outputs the types of object and its coordinates. & See C2.0 The Intel NUC model uses CPU and integrated GPU with must lower power without sacrificing performance. The mini pc is a cheap choice to achieve the performance requirements without using excessive power and computation.

CNN model storage The models are trained on the HP laptop or on Google Colab. The trained model is downloaded and stores in the Intel NUC mini pc. Large models consumes more power and causes more heating. Requiring more energy and cooling power. Training with large dataset and model is energy expensive and increases CO2 footprint.

The models are stored as compressed .keras files to reduce storage size.

State machine algorithm This algorithm integrates all ros2 nodes from each software sub-systems. Multiple programing running in parallel can adds computation overhead as a queue of processes/- commands are waiting to be executed. The use of state machine ensures only a specific parts of the sub-system is utilized to prevent redundant and saves energy. The use of server-client or publisher/- subscriber interfacing ensures the programs are only triggered when they need to be.

Intel NUC to Mycobot The Intel NUC mini pc computes the labels and coordinates data of the detected cylinders and ran a ros2 publisher node which sends these data to the Raspberry Pi model on the manipulator via ethernet cable. The manipulator receives them using a subscriber node. The publisher is periodically executed and adds up to the total compute power. Multiple data type exists and need to be send to the subscriber sequentially. This is limited by the bandwidth of ethernet connection. The publisher is only called when object data is available and the subscriber is only triggered once in the presence of available data. The data received by the NUC is combined into a single ros2 message, thereby reducing communication overhead and improves data transmission rate.

Depth Camera to Intel NUC The Intel NUC mini pc receives image data from the depth camera. This data is used by the deep learning model to make inference. The model is called periodically to update object label and coordinates. Image data are expensive to store. The CNN model uses low resolution to reduce storage spaces. This also allows for faster inference and reduced compute energy and cost.

2.3 Energy

Rover mobile base (Primary system) The rover's mobile base is the primary system of the whole robot's electrical power system (See Power Connection Diagram). It consists of the Leorover battery, the Leorover base (including 4 motors), the LeoCore (which consists of the Leorover controller and the Raspberry PI 5 model), the RGB camera and the wireless module (the Wifi adaptor and antenna). On average, the primary system requires a 20W power supply. Assuming the total operating duration of 30 minutes, the primary system consumes on average a total of 10Wh of electrical energy. Lithium batteries tend to be more sustainable for its recyclability and large energy-to-weight ratio. The rover base is made of light, recyclable materials to reduce operation power. The Buhler brushed motors uses encodes to improve communication efficiency and reduce power loss.

Rover on-board components The rover's on-board components forms the secondary system of the whole robot's electrical power system (See Power Connection Diagram). It consists of the Intel NUC mini pc, the Mycobot 280 PI manipulator, the Lidar, the depth camera. On average, the primary system requires 150–186W of power supply. The main main power demand coems from the Intel NUC mini pc and the manipulator. On average, the secondary system consumes 75–93Wh of electrical energy assuming a 30 minute operation duration. Lithium Ion power bank would provide enough energy with a low cost and still be sustainable. The manipulator is running on trigger enabled functions, making it energy efficient. The CNN model is only called when data is available and the Lidar data stored so that it can be reused by the navigation algorithm.

2.4 Waste*

Acrylic Board The CNC-cutted board used to mount all the on-board components on the rover. CNC-cutting can produce waste if the shape design is not space efficient. The component board is rectangular shaped with regular holes at fixed intervals. This makes it easy to manufacture and minimizes waste. Acrylic is also highly recyclable (See C1.3).

3D-printed parts The motor holder and the cylinder storage bins are 3D printed using PLA. 3D-printing can produce waste if the design contains fine details that creates frizzes or during re-printing if the designed parts are found to be flawed. Frizzes need to be grided smooth, creating potential micro-plastic pollutions. Failed parts also need to be disposed of. 3D-printing PLAs are widely recyclable. Also the components are designed to have large tolerance in design flaw so that manual adjustment can easily be implemented without having to re-print the whole part.

Electrical components (Excluding the power source) The electrical motor, the electronic circuit board (IC circuits) and electric wires. Moreover: 1. Wiring contains plastics and can create pollution if disposed incorrectly. 2. Wiring contains metals that can cause injury. 3. Circuit board contains plastics and silicon that can damage environment if disposed in landfills. Metals in wires, motors and circuit board are recyclable. The plastic components all need to be recycled properly to reduce waste. Ensures no additional wiring is used except essential power and data connections to minimize wiring use.

Battery The main battery and the power banks for the rover system. Lithium is toxic and harms the eco-system. The battery casing and wiring contains plastics. To minimize waste, ensure all the battery components are disposed properly in the correct recycling bin. Ensure the battery is only charged before operation and ensure no over charging to maximize its battery life span.

***Will only consider manufacturing waste as the rover does not produce additional waste during its operation**

2.5 Emissions

Tyre The rubber tyre attached to the 4 wheels. During rover's operation, the friction between tyre and ground leaves trails of rubber particles that can negatively impact soil, air quality and aquatic life. Can replace rubber wheels with plastic wheels to reduce pollutant emitted during operation.

2.6 Communications

Depth camera to Intel NUC Captures video of the environment as a sequence of image data to detect cylinders. Video frame rate: 30 FPS. Image resolution 640×640 pixels, 9.83 Mbits (uncompressed)/1.23 MB. Image data transmission rate: 295 Mbps or 36.8 MB/s. Image storage size: 38 kB (JPEG format).

The images are downscaled to include sufficient details for the object detection model to perform effectively without adding too much storage to the hardware. Only 1 frame is stored on the system per second with the new frame overwriting the old one as soon as the model inference has been completed for that image. These design minimizes storage space while ensures the model inference speed is sufficiently fast. The USB 3.0 connector that the camera uses is capable of transmitting 500-600 Mbps, which is able to handle the image data transfer rate.

Lidar to Intel NUC Scans the environment and produces a 2D point cloud map of the rover's surrounding objects. Scan rate: 5-15 Hz, typical 10 Hz. Scanning frequency: 16 kHz. Max. points per second (assuming no scan loss or missed scan): 16000 points/seconds. Data recording rate: 65.5 Mbps or 8.19 MB/s (assuming each data point is represented as a tuple of 64-bit magnitude and a 64-bit angle).

Every Lidar scan produces a point cloud map that is stored permanently on the Intel NUC until mission terminates. This is essential for the accurate computation of the SLAM algorithm for rover navigation.

The USB 3.0 connector that the Lidar uses is capable of transmitting 500-600 Mbps, which is able to handle the image data transfer rate. The Lidar data is represented as rows containing useful information such as point distance and angle, as well as redundant information such as scan rate/time or statistical readings such as min./max. values of measurements. The navigation algorithm is designed to only use the point information. This trade-off increases data transmission overhead as only a fraction of information received is useful, but lowers storage space by only storing relevant information in the data.

2.7 Modularity

Base rover & Components (Aluminum CNC-cutted haul, 3D-printed motor mount, acrylic modular board, motors, Raspberry Pi 5, Lidar, storage bin, battery, secondary system.)

The rover base is made from CNC cut metal board or 3D-printed components, these are durable and easy to replace when missing or damaged. The aluminum haul and acrylic board has arrays of component mounting holes drilled at regular intervals. This offers a great design flexibility in terms of placing components onto the rover. The storage bin is also 3D-printed and easily replaceable. The battery can be easily replaced through laboratory order or online purchase. The battery and the Raspberry Pi 5 model each have its mounting box, which allows them to be mounted at arbitrary locations beneath the rover. Any additional/new components can also be mounted at any location on the rover.

Secondary system (MyCobot manipulator, Mycobot adaptive gripper, Intel NUC mini pc and depth camera.)

The manipulator, Intel NUC and the depth camera can be mounted at arbitrary locations on the acrylic board. The adaptive gripper can be easily replaced with other types of gripper. Any new components such as new/second manipulator or a second camera can also be easily added onto the acrylic board's modular mounting holes.

Computer vision (Software components: *CNN model, object info. publisher, ros2, real time image data.*)

CNN models are pre-trained in cloud platforms before loaded into the NUC. Any pre-trained deep learning models can be easily integrated with the system using yolo library. This ensures users with no background in computer vision can easily operate this robot fully autonomously. The publisher is a custom ros2 node written in Intel NUC. Any new custom ros2 node can be integrated with the Intel NUC and the manipulator as long as they have the same topic name.

Manipulator control (Software components: *pymycobot, object info. subscriber, ros2*)

For any Mycobot series manipulators, the same library can be installed and used directly using our code. The manipulator uses a custom ros2 node which subscribes to any messages published to it from the NUC via the same topic. Any custom ros2 message can be defined and send to the manipulator for control purpose. This allows easy learning and re-use of the robot with a new Mycobot manipulator.

Navigation (Software components: *Rviz, ros2*)

The navigation software uses built-in SLAM solver. The input of the navigation system can be environment map generated by any sensor such as Lidar or RGB cameras. The navigation program is ran directly on Intel NUC and requires no additional setup. This means any new user only needs to attach their sensor and the robot will automatically navigate the environment.

State machine (Software components: *Computer vision program, manipulator program, navigation program, ros2.*)

The state machine is responsible for the high level decision making process of the rover as well as integrating all software sub-modules. & The state machine receives feedback inputs from each of the other software sub-systems. Since each software sub-system (including the state machine itself) can be directly used or installed to the rover with minimal modification, the entire software integration can be installed as a single library to the rover. This includes the manipulator's control software package which does not depend on any external system and can be installed and used without any compatibility issues. This assists any future researchers by having all software packages ready to use.

2.8 Location/Placement

The robot is designed to operate within a controlled indoor environment, specifically a 6 m × 4 m search area used for testing and evaluation. As the robot is compact and self-powered, it can be easily transported by hand between laboratory environments without requiring dedicated transportation infrastructure.

Because the robot is battery powered and untethered, it can be deployed in different indoor environments without requiring modifications to existing facilities.

2.9 Maintenance

Maintenance considerations were incorporated into both the hardware and software design.

Mechanical components are mounted using bolted connections, allowing individual parts to be removed and replaced easily. Electronic modules such as the Intel NUC, Raspberry Pi boards, and sensors are mounted using accessible brackets and clamps to simplify replacement.

Software diagnostics can be performed through ROS2 monitoring tools, including RQT graphs and node diagnostics, which allow developers to observe system performance and identify potential faults.

Routine maintenance tasks include:

- Battery health monitoring
- Mechanical fastener inspection
- Sensor calibration checks
- Software log inspection

These measures help ensure reliable operation over the robot's lifetime.

2.10 Repurposing

At the end of the project, the robot platform can be repurposed for future robotics research and educational applications.

The Leo Rover platform, perception sensors, and manipulator arm are all general-purpose robotic components that can be reused for other tasks such as:

- autonomous navigation research
- mobile manipulation experiments

- robotics education and laboratory demonstrations
- perception and machine learning experiments

Because the system is built using modular hardware and open-source ROS2 software, the platform can easily be adapted to new research problems without requiring significant redesign.

3 Cyber Security Considerations

The robot uses multiple onboard computing and communication components, including an Intel NUC, a Raspberry Pi 5, a Raspberry Pi 4B, a wired Ethernet link and a Wi-Fi interface. As a result, cyber security should be considered as part of the overall system design rather than only as a network issue.

A key risk is unauthorised access to onboard computers or control interfaces. If weak credentials, default accounts or unrestricted remote access are used, an attacker could interfere with mission execution. A second risk is command tampering or unsafe triggering of actions, since the software stack coordinates navigation, grasping and sorting through ROS-based nodes. A third risk is loss of availability caused by communication failures, software crashes or denial-of-service conditions.

To reduce these risks, access should be limited to authorised team members, default credentials should be avoided, and unnecessary services should be disabled. Software dependencies and system packages should be kept updated, and important access or fault events should be logged where practical. In addition, if communication or critical nodes fail, the robot should enter a safe state, such as stopping motion and cancelling active actions, rather than continuing with stale commands.

Although cyber risk cannot be removed completely, these measures reduce the likelihood of compromise and limit the impact on robot safety and mission performance.

4 Updated Preliminary Design Review

4.1 System Design

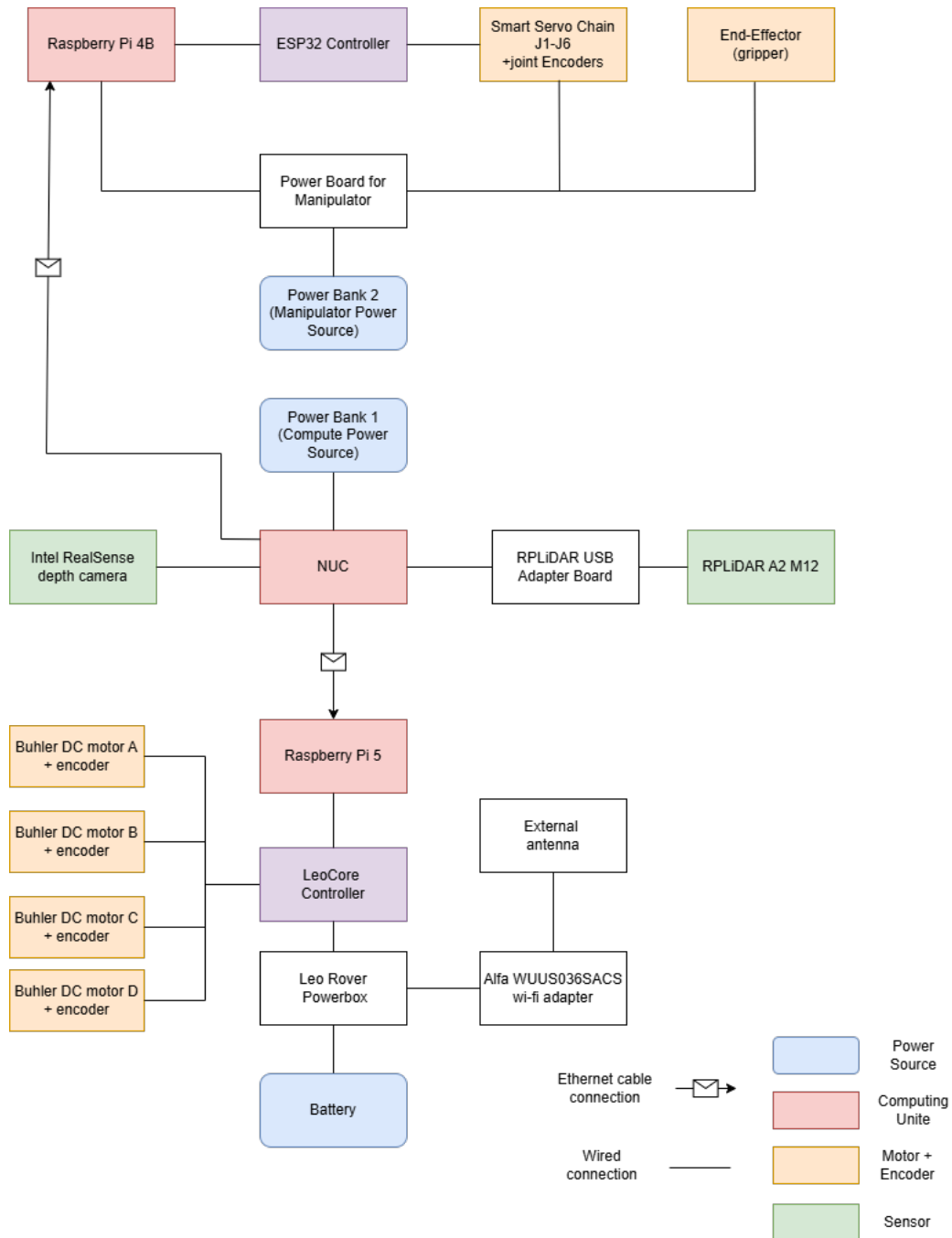


Fig. 1. Complete system architecture of the autonomous rover and manipulator platform.

4.1.1 System Components and Justification

The final hardware architecture combines sensing, computation, actuation, communication and power delivery in a structure suitable for reliable real-time operation. Each component in the system fulfills a specific role that is required for correct functionality, integration and performance during the 20-minute autonomous mission. Below, all components are described together with the justification for their inclusion.

Computing Units:

Intel NUC (Primary Compute Unit): The NUC is the main processing platform for the system. It runs all high-level ROS 2 nodes, including perception, LiDAR processing, object detection, mapping, and manipulation control. These tasks exceed the computational capability of embedded boards, so the NUC is required to guarantee real-time performance and unify the software stack on a single central machine. By including the NUC in the design, the rover can handle all computation locally, thereby satisfying FR12.

Raspberry Pi 5 (Leo Rover Host): The Pi5 interfaces with the LeoCore motor controller and acts as the rover's internal control computer. It runs the rover firmware, publishes wheel odometry, and provides the API used by the rest of the system. The Pi5 is required because the Leo Rover electronics are designed around it.

Raspberry Pi 4B (Manipulator Host): The MyCobot 280 manipulator requires a compact Linux board to interface with the ESP32, handle USB communication, and run the manufacturer's ROS packages. The Pi 4B serves as this dedicated control host and ensures the manipulator subsystem operates independently while still communicating with the main NUC.

Controllers:

ESP32 Controller: The ESP32 communicates directly with the smart servos, handles low-level joint commands, and streams encoder feedback. It provides deterministic timing and simple communication pathways that are required by the manipulator's control protocol.

LeoCore Controller: This module drives the four DC motors, reads wheel encoders, and provides odometry to the Pi 5. It is responsible for all low-level rover motion control and is necessary for safe and reliable operation of the Leo Rover drivetrain.

Sensors:

Intel RealSense Depth Camera: The depth camera provides RGB-D data needed for object detection, distance estimation, and visual guidance of the manipulator. It is also used for environment perception tasks where depth information is essential. This sensor directly supports FR6 by enabling accurate detection and localisation of target objects and FR8 by allowing the robot to identify the correct sorting bins.

RPLiDAR A2M12 and USB Adapter Board: The 2D LiDAR provides planar distance data used for localisation, obstacle detection, and mapping. It is a requirement for the navigation portion of the project which directly addresses FR1 (autonomous navigation), FR2 (collision avoidance), FR3 (obstacles detection and mapping) and FR4 (self-localisation). The USB adapter board ensures stable power and communication, which is necessary for reliable LiDAR performance.

Actuation:

Smart Servo Chain J1–J6 with Encoders: These servos provide all six joints of the manipulator with position feedback and closed-loop control. Encoders are essential to achieve accurate joint angles and repeatable motion.

End Effector (Gripper): The gripper is required for interaction with objects, enabling the system to perform manipulation tasks such as picking and placing

Bühler DC Motors A–D with Encoders: These motors power the rover's wheels. The encoders provide odometry for navigation and closed-loop control. They are essential for movement and localisation.

Communication Components:

Ethernet Cable Connection: A direct wired connection between the NUC ,the Pi 4B on the manipulator and Pi 5 on the Rover ensures stable, low-latency communication with the manipulator subsystem. Wired communication avoids packet loss and timing uncertainty, which is important for real-time control.

Alfa AWUS036SACS Wi-Fi Adapter: Provides reliable wireless connectivity, allowing the robot to transmit and receive data, commands, and software updates throughout development and testing.

Power system components:

Leo Rover Battery: The main rover battery powers the drivetrain (DC motors), the LeoCore controller, the Pi 5, and onboard networking components. It is essential for all base rover operations. Powering the rover with this onboard battery satisfies PR9.2.

Power Bank 1 (Compute Power Source): Power calculations showed that the rover battery alone cannot supply both the drivetrain and the high-load components such as the Intel NUC for the full mission duration. A dedicated power bank is therefore used to power the NUC through a 20 V trigger cable, ensuring that the compute subsystem receives a stable and sufficient power supply. Separating the compute power from the rover's drivetrain power helps avoid voltage drops and prevents performance degradation during high computational loads. This configuration ensures that the robot can reliably perform perception, mapping, and manipulation planning tasks throughout the mission while maintaining sufficient energy capacity for the entire operational period. Providing a dedicated onboard power source for the compute subsystem also contributes to satisfying PR9.1 and PR9.2 by ensuring reliable untethered operation.

Power Bank 2 (Manipulator Power Source): A second power bank is used to supply power to the manipulator subsystem via a 12 V trigger cable. The manipulator's servo motors and control electronics require a stable power supply that can handle dynamic load variations during arm motion and object manipulation. Supplying this subsystem through a separate power bank isolates the manipulator power demand from both the rover drivetrain and the compute subsystem, reducing the risk of voltage fluctuations that could affect system stability. This separation of power systems improves reliability during manipulation tasks and guarantees sufficient energy capacity for the duration of the mission. As with the compute subsystem, powering the manipulator from an onboard rechargeable power bank ensures that the robot operates fully untethered, satisfying PR9.1 and PR9.2.

Power Board for Manipulator: This board distributes power to the, Raspberry Pi 4B, smart servos and provides stable voltage to the manipulator subsystem. It is required to handle the current draw of multi-joint motion and protects the upstream power source.

4.1.2 Summary

Each component in the system plays a defined role in sensing, computation, actuation, or communication. The NUC serves as the central compute unit due to the computational requirements of perception and manipulation tasks. Additional power was added to ensure sufficient runtime and electrical stability. Together, the components form a robust architecture that supports the rover's autonomous and manipulation capabilities, equipping it with the necessary functionalities to meet the overall objective of completing the search, retrieval and sorting mission within 20 minutes (FR11).

4.2 Mechanical Design

4.2.1 Design Overview

Figure 2 shows a labelled diagram of the robot's CAD model containing the Leo Rover and a payload sled which holds the LiDAR, Next Unit of Computing (NUC), camera, manipulator, 2 Anker power banks and an onboard storage bin.

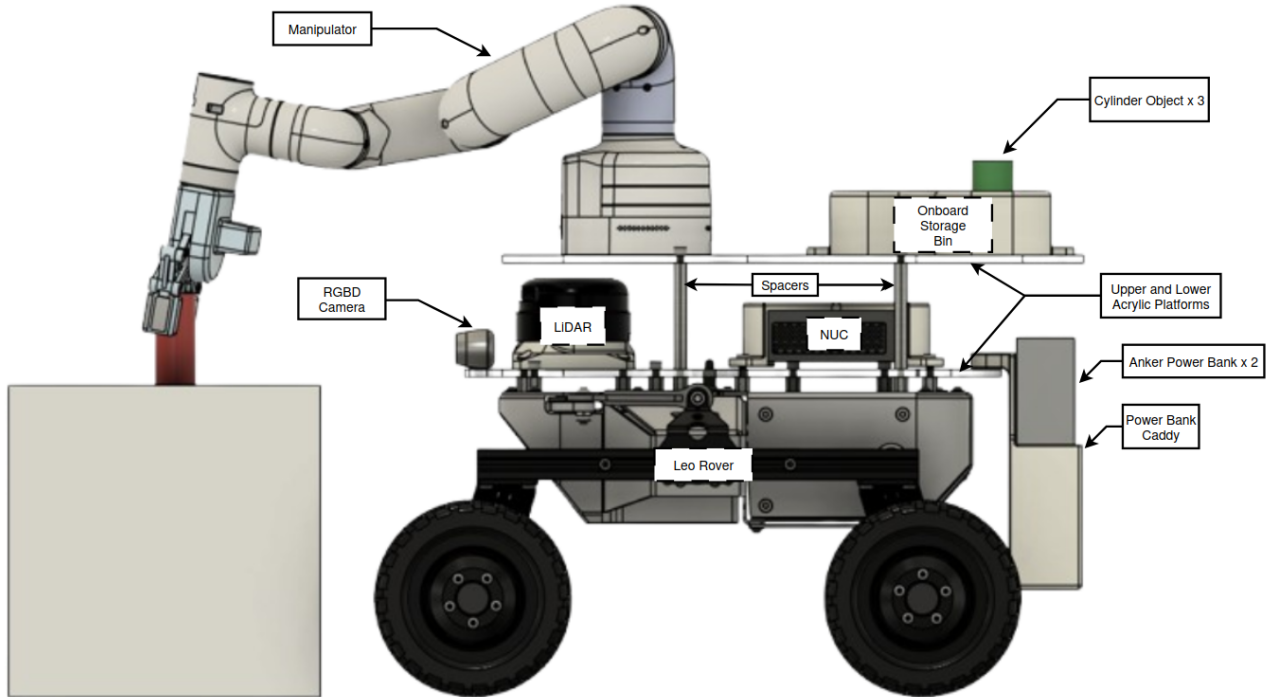


Fig. 2. Labelled diagram of the robot.

The payload sled consists of two 5mm thick acrylic platforms separated by 70mm spacers as shown in Figures 3 and 4. The base platform holds the depth camera at the front, the LiDAR on an elevated 3D printed holder, a NUC secured in place using a 3D printed clamp and a 3D printed power bank caddy which holds the 2 Anker power banks. The upper platform holds the manipulator at the front and an onboard object storage bin at the back.

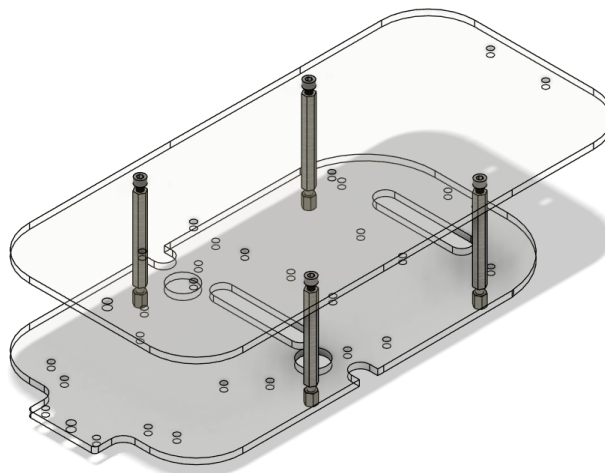


Fig. 3. Preview of the payload sled platforms.

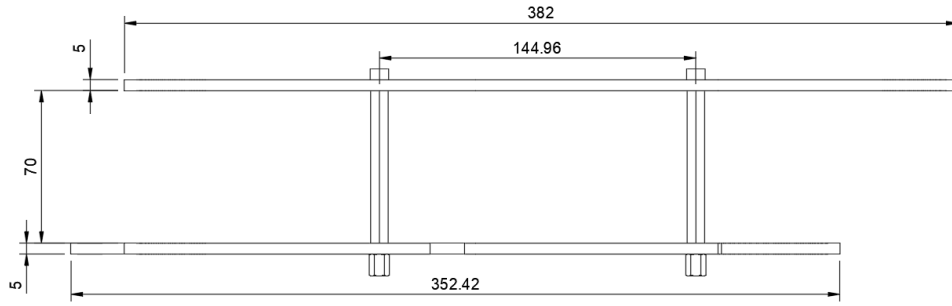


Fig. 4. Dimensions of the payload sled from side view.

4.2.2 Design Considerations

When designing the acrylic base platform, extra care was taken to maintain access to critical components on top of the rover such as the antenna, micro-USB port and the 6 bolts which secures the battery and power boxes. Mounting holes are cut out for the depth camera, LiDAR holder, NUC clamp and the power bank caddy. Additionally, some access slots were created near the NUC to tuck away excess cables in the 10mm spacing between the base platform and the rover. To minimise flexing caused by reaction forces generated during manipulator motion, the base platform is secured to the top of the rover using four pairs of bolts along its length. These bolts fasten the platform to 10 mm spacers, which are in turn bolted to the rover chassis. The design of the upper platform is comparatively simpler, featuring attachment points for the manipulator, the storage bin and the four spacers. Figures 5 and 6 shows the design and dimensions of the base and upper platforms.

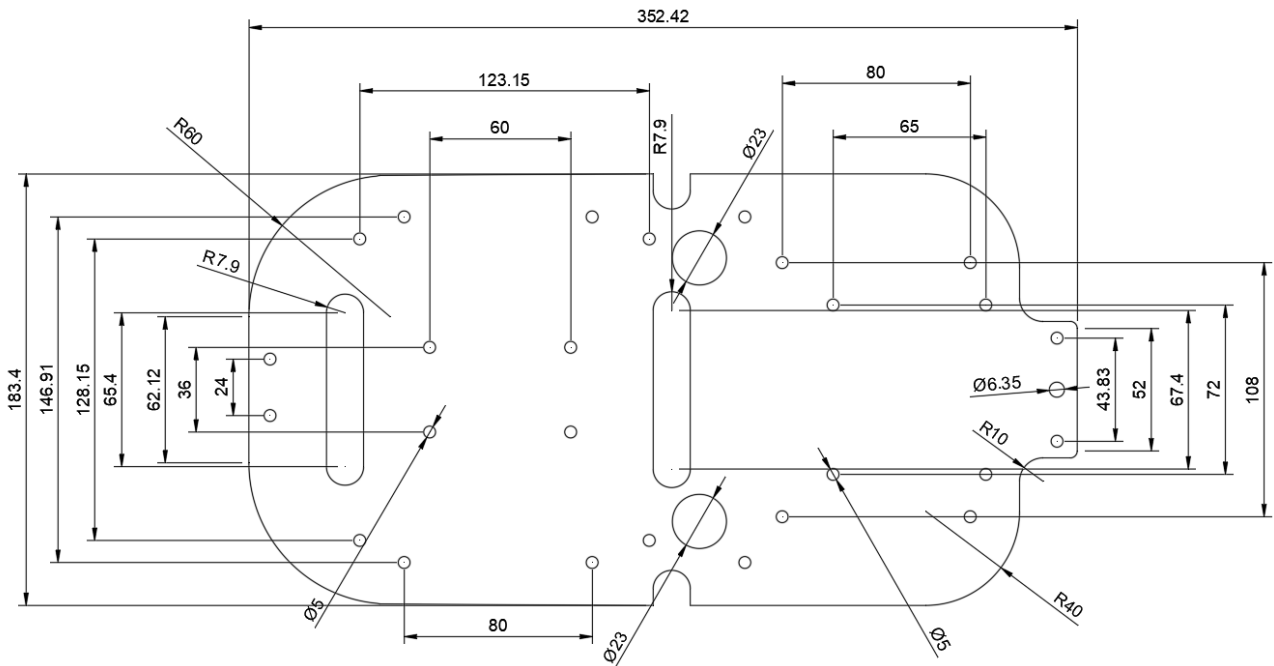


Fig. 5. Dimensions of the base platform.

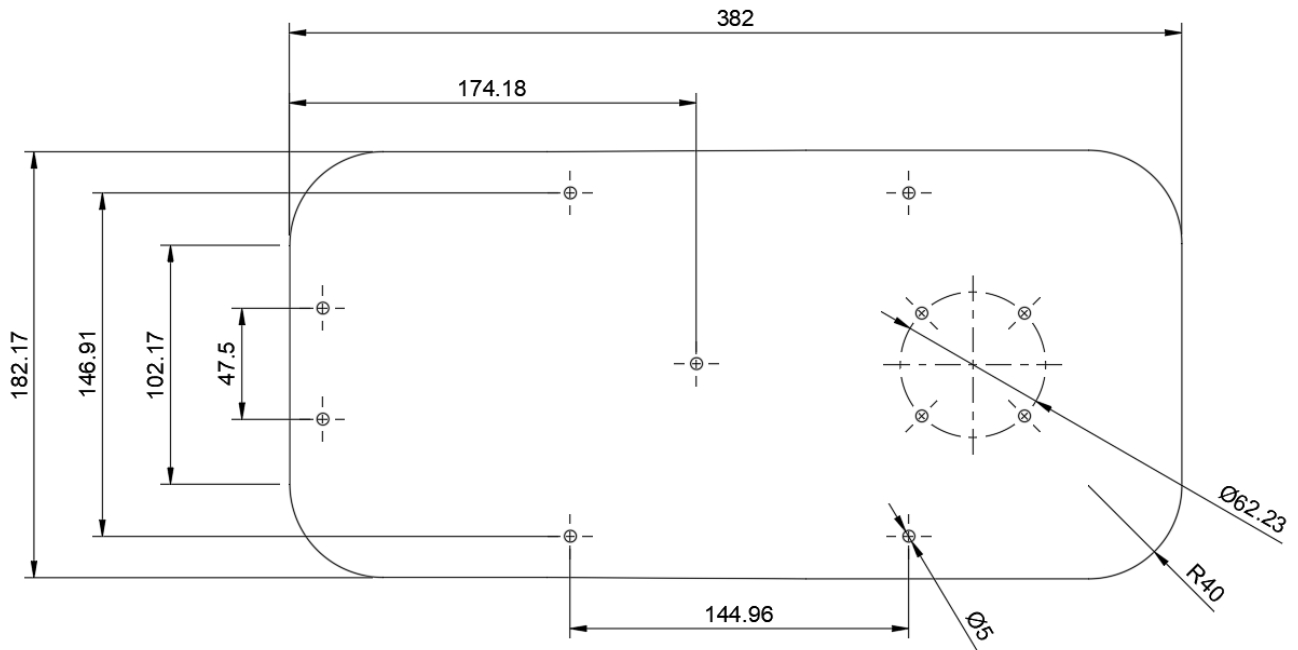


Fig. 6. Dimensions of the upper platform.

The depth camera is placed at the front of the base platform to provide a better view of the objects that it must detect. Placing the camera on the base platform means that it is level with the height at which the object is placed. If the camera is placed on the upper platform, some amount of tilting is required to better capture the object at close distances. It was decided that no support structures will be placed 180 degrees in front of the LiDAR to provide an unobstructed field of view ahead of the robot. This design choice aligns with FR3 as it prevents occlusion, enabling the robot to accurately detect and map obstacles in its field of view. The NUC is held in place using a 3D printed clamp that only covers its corners, maintaining accessibility to all its peripheral connection points. Figure 7 shows the depth camera, LiDAR and NUC.

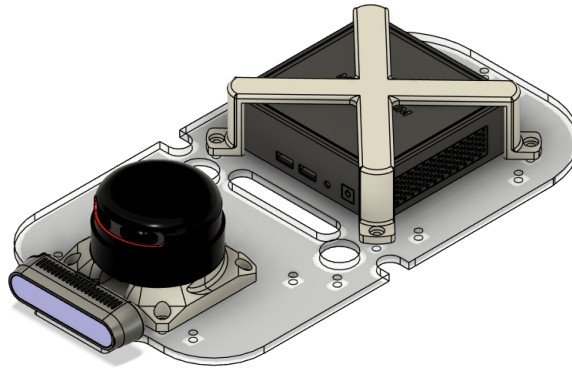


Fig. 7. Design of depth camera, LiDAR and NUC.

The onboard storage bin shown in Figure 8 was designed to hold 3 cylinders of 26mm diameter and 60mm height. It was decided that the storage space for these cylinders would have dimensions of 35 x 35 x 40 mm. This leaves enough of the cylinder exposed for the gripper to comfortably grasp it during retrieval from the storage bin. This also leaves enough tolerance for the manipulator to place the cylinder in the storage bin while still being able to hold the cylinder upright, in a predictable position. This is advantageous for picking the cylinder from the storage bin during the sorting process as the pose of the cylinder is fixed. The edges of the storage space are also angled at 30 degrees so that the cylinder can slide into the bin if the manipulator is slightly off-centre. As 66% of the cylinder is within the storage space, it is very unlikely that it would topple out of the onboard storage bin during the mission, especially when considering that the robot will be operating on a flat and even terrain. These design choices directly address FR7 and PR7.1.

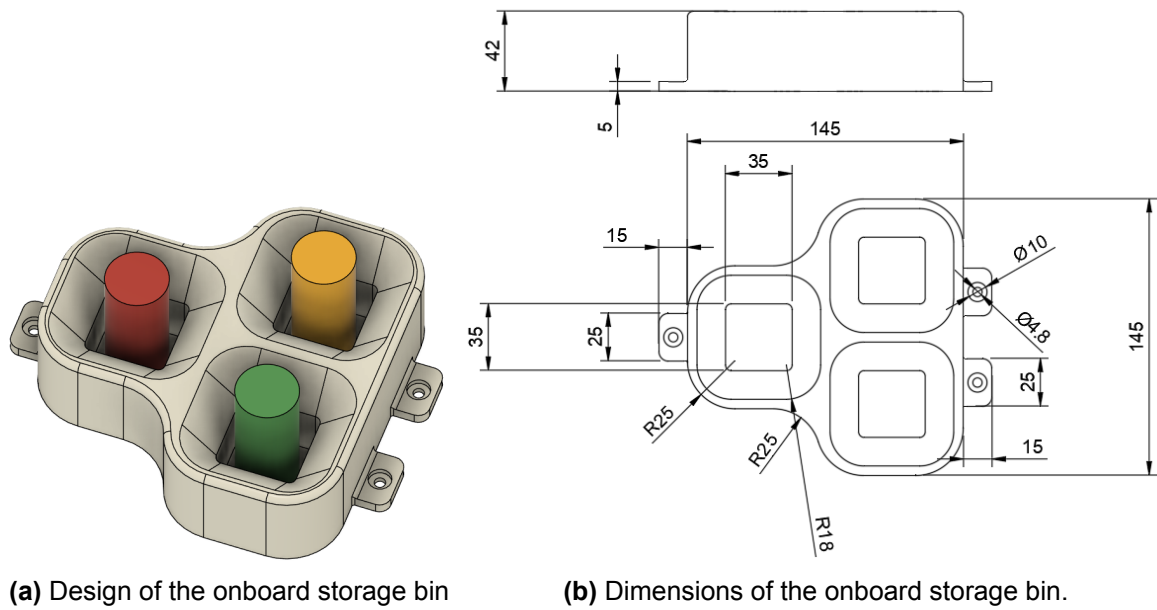


Fig. 8. Onboard storage bin

As two Anker power banks were chosen to power the NUC and the manipulator, they must also be accommodated by the payload sled. This is done using the power bank caddy which is attached to the back of the base platform as shown in Figure 9. These power banks are positioned in such a way that they do not intercept the LiDAR's scanning plane.

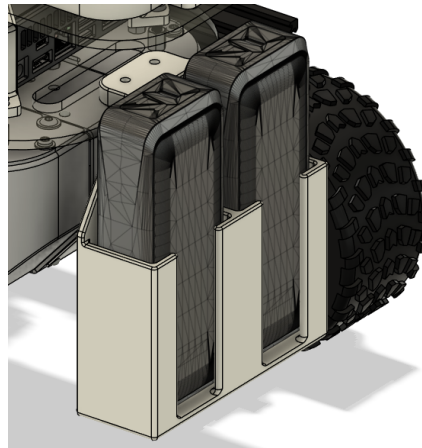


Fig. 9. Design of the power bank caddy.

4.2.3 Component Placement Considerations

The LiDAR is placed on the base platform at the front of the robot to increase its maximise its field of view and its chances of detecting obstacles. The 3D printed LiDAR holder was slightly modified from the version provided by fictionlab, its height was increased by a 5mm to raise the LiDAR such that it could see over the NUC towards the back of the rover. This increased the LiDAR's field of view, allowing the rover to map more of its environment in a single scan, maximising the robot's ability to satisfy FRs 1-4.

The camera is placed at the front of the base platform to provide the best view of the target objects in the search environment, maintaining a large field of view and increasing its chances of detecting the target object, addressing FR5.

The manipulator is placed on the upper platform. Keeping the manipulator towards the front improves its ability to access the target objects during pickup without colliding with the upper platform. This

ensures that the robot can successfully pick up the target object consistently, enabling it to meet PR6.1. Figure 10 shows the pose of the manipulator when picking up a target object.

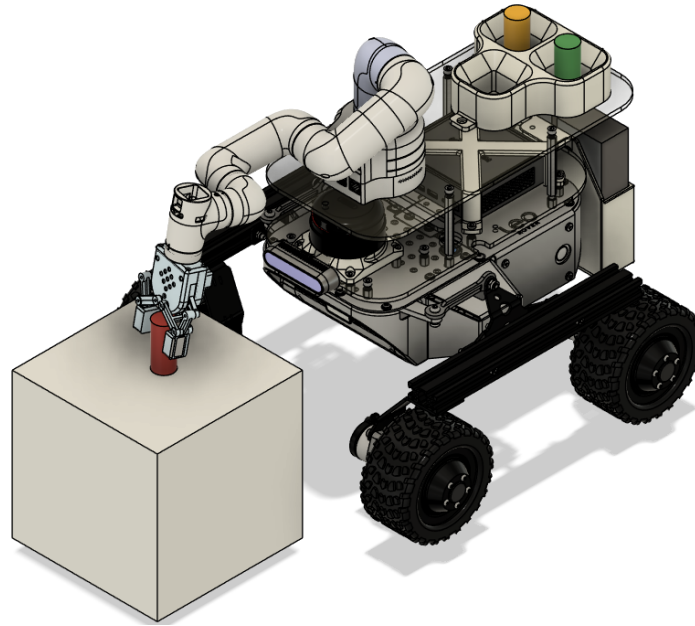


Fig. 10. The manipulator pose when picking up cylindrical object from the box that it is placed on.

The storage bin is placed far enough from the manipulator to allow it to comfortably access the closest storage space. This improves the manipulator's likelihood of successfully placing cylinders inside the onboard storage bin's spaces, supporting compliance with PR6.2.

4.2.4 Material Choice and Fabrications

The base and upper platforms were manufactured from 5mm thick acrylic due to its suitability for laser cutting, enabling high dimensional accuracy, repeatability and rapid fabrication with minimal post-processing. Acrylic provides sufficient structural rigidity and strength-to-weight ratio to support the mounted components while remaining lightweight for mobile operation. From a manufacturability perspective, laser cutting allows efficient batch production and straightforward design iteration through simple CAD modifications. Although aluminium could have been used as an alternative, offering greater strength and durability, it would require more complex and costly manufacturing processes such as CNC machining, making it less suitable for rapid prototyping and iterative development.

Components with more complex geometries, such as the LiDAR holder, NUC clamp, onboard storage bin and the power bank caddy were fabricated using 3D printed PLA. Additive manufacturing was selected as it enables the production of intricate shapes and customised geometries that would be difficult to achieve using subtractive methods. PLA is cost-effective and widely available, making it appropriate for rapid prototyping and low-volume production. This approach also improves manufacturability by allowing integrated features such as mounting points and fastening interfaces to be incorporated directly into the design, eliminating the need for additional parts.

4.2.5 Strength

A finite element analysis was conducted to study the stress and deformation of the upper platform under the weight of the camera, manipulator and storage bin. The weight of these components was simulated as a distributed pressure acting on an area equivalent to their footprint on the platform and the locations where the standoff spacers connect to the platform were set to have fixed constraints.

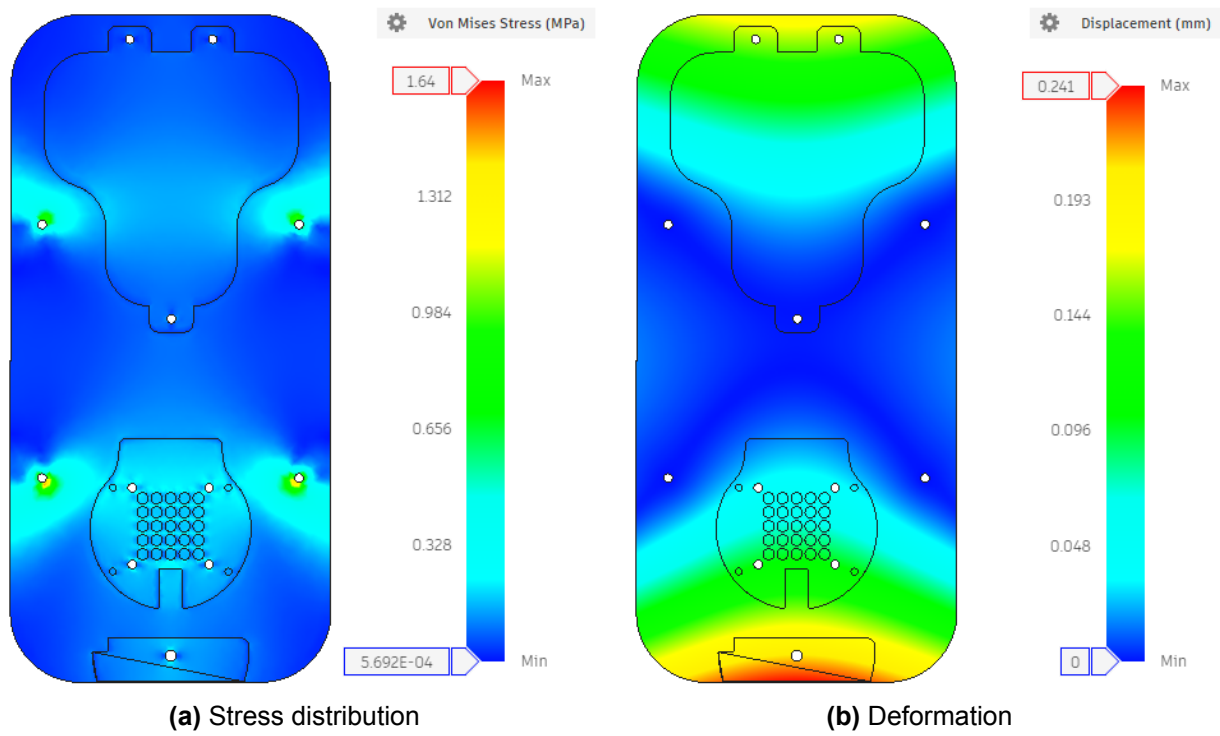


Fig. 11. Finite Element Analysis of the upper platform.

The stress distribution results show that the maximum stress observed is 1.64 MPa near the points where the standoff spacers meet the platform. This is well under the 70 MPa failure stress of clear acrylic material. A maximum deformation of 0.241mm is observed at the front of the upper platform which is also negligible compared to the size of the platform. As no significant sagging was observed in the FEA simulations of the upper platform, it can be concluded that the platform has sufficient strength to hold the camera, manipulator and storage bin. These results directly support the requirement that the robot shall maintain mechanical stability (FR10).

4.2.6 Stability

Since the Anker power banks weigh 600g each and as they are added to the rear of the rover, it was necessary to check that the combined weight of the two power banks are not enough to cause the rover to tip backwards. This was done using a simple moment comparison around the tipping point which was considered to be the contact point between the rear wheel and the ground as shown in Figure 12. Only the rover's mass (6.5 kg) is being considered for this calculation as it far outweighs all other components on the left side of the tipping point.

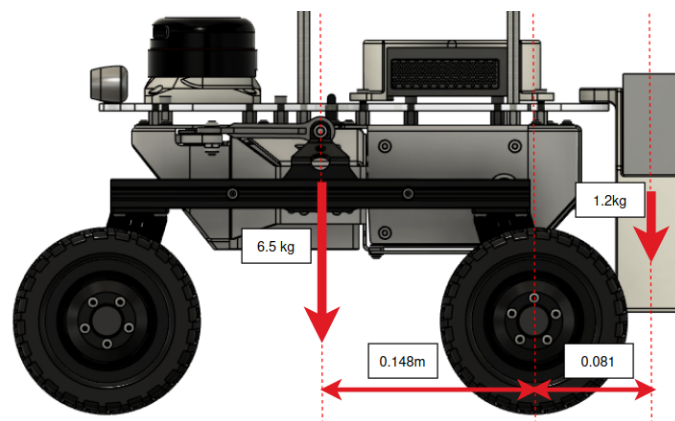


Fig. 12. Diagram used for moment calculation.

To the left of the tipping point, there is a moment of 9.437 Nm created by the rover whereas to the right of the tipping point, there is a moment of 0.954 Nm created by the power banks. Since the moment created by the rover is 10 times greater than the moment produced by the power banks, it is not possible for the rover to tip due to the addition of the power banks.

4.3 Electrical Design

4.3.1 Power Connection Diagram

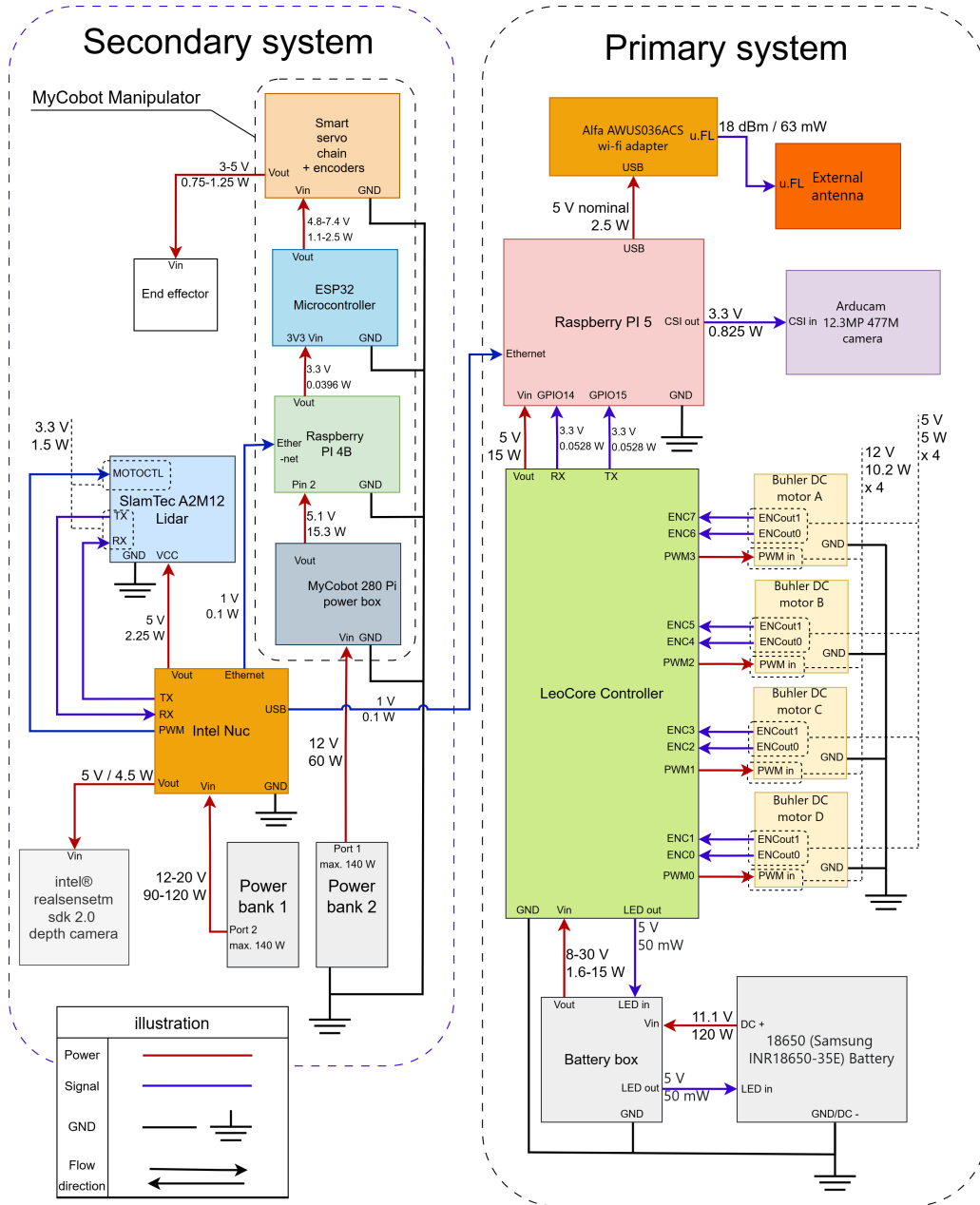


Fig. 13. Power connection diagram.

The power attributes of the manipulator's servo motors were estimated using the rated torque, stall torque and the maximum joint angular velocity.

The electrical diagram includes the typical voltage and power value of each sub-component and ports.

The gripper's power attributes were estimated using two similar products for the myCobot 320 gripper models [14].

The power attributes of the manipulator's servo motors were estimated using the rated torque, stall torque and the maximum joint angular velocity [18]. The power can be calculated by using the torque and the angular velocity of the motor joints assuming 100% efficiency:

$$P = \tau \times \omega \quad \text{assuming } P_{\text{mech}} \approx P_{\text{elec}} \quad (1)$$

The rated current can be calculated as $I = \frac{P_{elec}}{V}$

From that we obtained: Rated power: 2.5W, Stall power: 1.14-1.23W.

The voltage and power values of all other electrical components were obtained directly from their datasheet or the corresponding online sources.

4.3.2 Power Budget Calculations

The power is calculated based on typical current usage. This was done assuming all devices are powered on simultaneously at constant rate. In real-life, the rover's total power drawn is highly dynamic and varies with time. Many electrical characteristics such as the transient effect, the frequency/phase response or the PWM duty cycle can change how much power the system uses over time. Hence, to calculate the power demand of the base rover (without the on-board manipulator and intel NUC), we used the power supplied by the battery and assume the base rover would be continuously operating for 4 hours. We then obtained a time average power usage of the base rover to estimate the actual power consumption. Apart from the base rover system, the on-board system consists of the manipulator, the intel NUC, the lidar and the depth camera can be estimated more easily as they have an operation power rating and requires constant power supply, making their power to be approximately time-constant. The power usage of the manipulator is relatively more time-varying so we used the minimum and maximum power for a more reliable estimation.

Table 1. Power buedge calculation

Item	Voltage Rating	Current Rating	Power	Electrical En- ergy (30 min.)
Primary system (Base rover)			20 W	10 Wh
Samsung Battery	3.6-3.7 V ($\times 4$)	5-25 A	120 W	77.7 Wh
Manipulator	12 V	5 A	24-60 W	
Anker Prime Power Bank (Port 1, USB-C)	5-28 V	3-5 A	max. 140 W	61.25-343 Wh
Lidar	8.3 V	2.65 A	3.75 W	
Depth camera	5 V	0.5 A	2.5 W	
Intel NUC	20 V	6 A	120 W	
Secondary sys- tem			150.25-186.25 W	75.13-93.13 Wh
Anker Prime Power Bank (Port 1, USB-C)	5-28 V	3-5 A	max. 140 W	61.25-343 Wh

The primary system consists of the LeoCore controller [19], the LeoRover RaspberryPI 5 board **<empty citation>** 4 Buhler motors (9), the Arducam camera (10), the Wi-Fi adaptor (11) and an antenna. This system is responsible for navigation and movement of the rover. The primary system is supplied by the Samsung INR185650 battery pack which has a maximum output power of 120W and a total electrical energy of 77.7 Wh (7). This would supply enough power to the base rover which demands only 10 Wh of electrical energy for 30 minutes operation duration. A battery box is connected to the battery and re-distribute the power from the battery to all the components in the primary system. The controller accepts the power supply from the battery box and then distribute it to all the other attached components. The Raspberry PI board also act as a power supply to all the components that are attached to it.

The secondary system is responsible for computer vision (mainly object recognition) and manipulation. It consists of the intel NUC, the myCobot 280 PI manipulator (with gripper attached) the lidar and the depth camera. The secondary system requires an additional power supply of 150.25-186.25 W or a total energy supply of 75.13-93.13 Wh. The manipulator system consists of a Raspberry PI 4B board (8), an ESP32 microcontroller (12), 6 metal servo motors (2) and the myCobot gripper (1). The manipulator has its own power box which will take the power supply from the additional power bank and distribute the power to all the connected components.

4.3.3 Proposed Solution

The power budget calculations address FR9. From the power calculations above, the battery provided is not able to supply the full electrical power or energy required by the entire system to perform the task for 30 minutes. However, it is in fact capable of supplying enough energy (assuming the operation time does not exceed 30 minutes) to the rover alone without add-on systems. The proposed solution uses an additional power source to power the manipulator as well as the NUC along with its sensors. The 120 W charging port of the power bank would be enough to power the intel NUC with required 120 W power supply. The 65 W outlet port would be able to supply enough power to the manipulator and the two remaining USB ports can supply enough power for the lidar and the depth camera. The power pack has total energy of 116 Wh (6) which is sufficient in supplying the secondary system which demands a maximum energy of $(60 + 30 + 6.25) / 0.5 = 93.13$ Wh (as the total task duration is 30 minutes the multiply factor is 0.5). Therefore, this proposed solution could supply more energy than the system requires and would enable the robot to meet PR9.1. As the robot is using an onboard lithium-ion battery and the remaining components are powered by a power bank, which is also mounted on the robot, it is not going to require any wired connections for power, meeting PR9.2.

Hence, we split the whole system into a primary system consists of the mobile base of the rover, and a secondary system which consists of the manipulator, the Intel NUC and all its sensors. The secondary system is powered by 2 additional power banks provided by the laboratory.

Power Bank 1 (Compute Power Source): Power calculations showed that the rover battery alone cannot supply both the drivetrain and the high-load components such as the Intel NUC for the full mission duration. A dedicated power bank is therefore used to power the NUC through a 20 V trigger cable, ensuring that the compute subsystem receives a stable and sufficient power supply. Separating the compute power from the rover's drivetrain power helps avoid voltage drops and prevents performance degradation during high computational loads.

Power Bank 2 (Manipulator Power Source): A second power bank is used to supply power to the manipulator subsystem via a 12 V trigger cable. The manipulator's servo motors and control electronics require a stable power supply that can handle dynamic load variations during arm motion and object manipulation. Supplying this subsystem through a separate power bank isolates the manipulator power demand from both the rover drivetrain and the compute subsystem, reducing the risk of voltage fluctuations that could affect system stability.

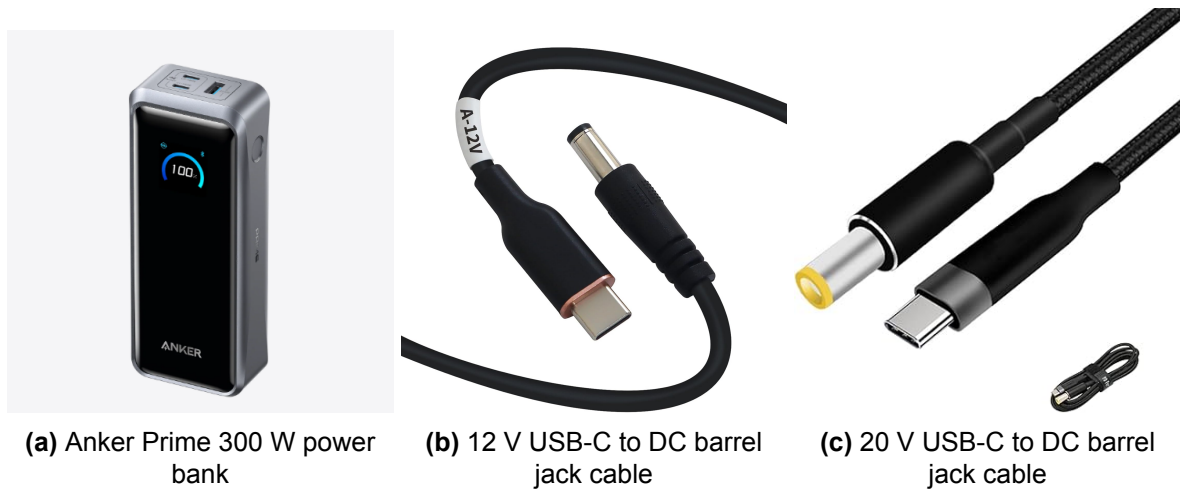


Fig. 14. Components of the proposed solution.

4.4 Software Design

4.4.1 Software Diagram

Figure 15 shows the software block diagram of the entire robot system. All of the computing occurs in the NUC using data from sensors in the Leo Rover, LiDAR, depth camera and MyCobot manipulator.

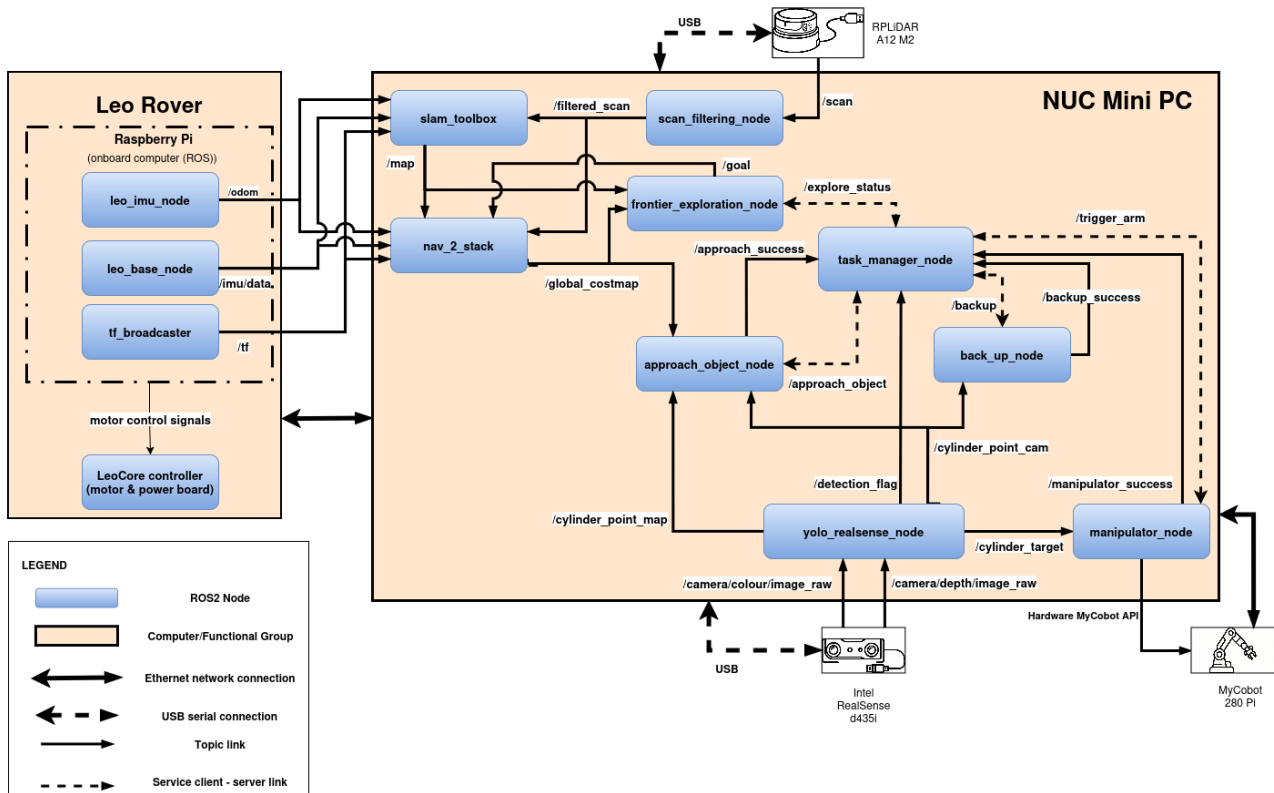


Fig. 15. Software block diagram.

4.4.2 State machine

The entire mission is coordinated by the task manager node which acts like a state machine and triggers other nodes when required during the mission. Figures 16 and 17 show the flow diagram for the state machine.

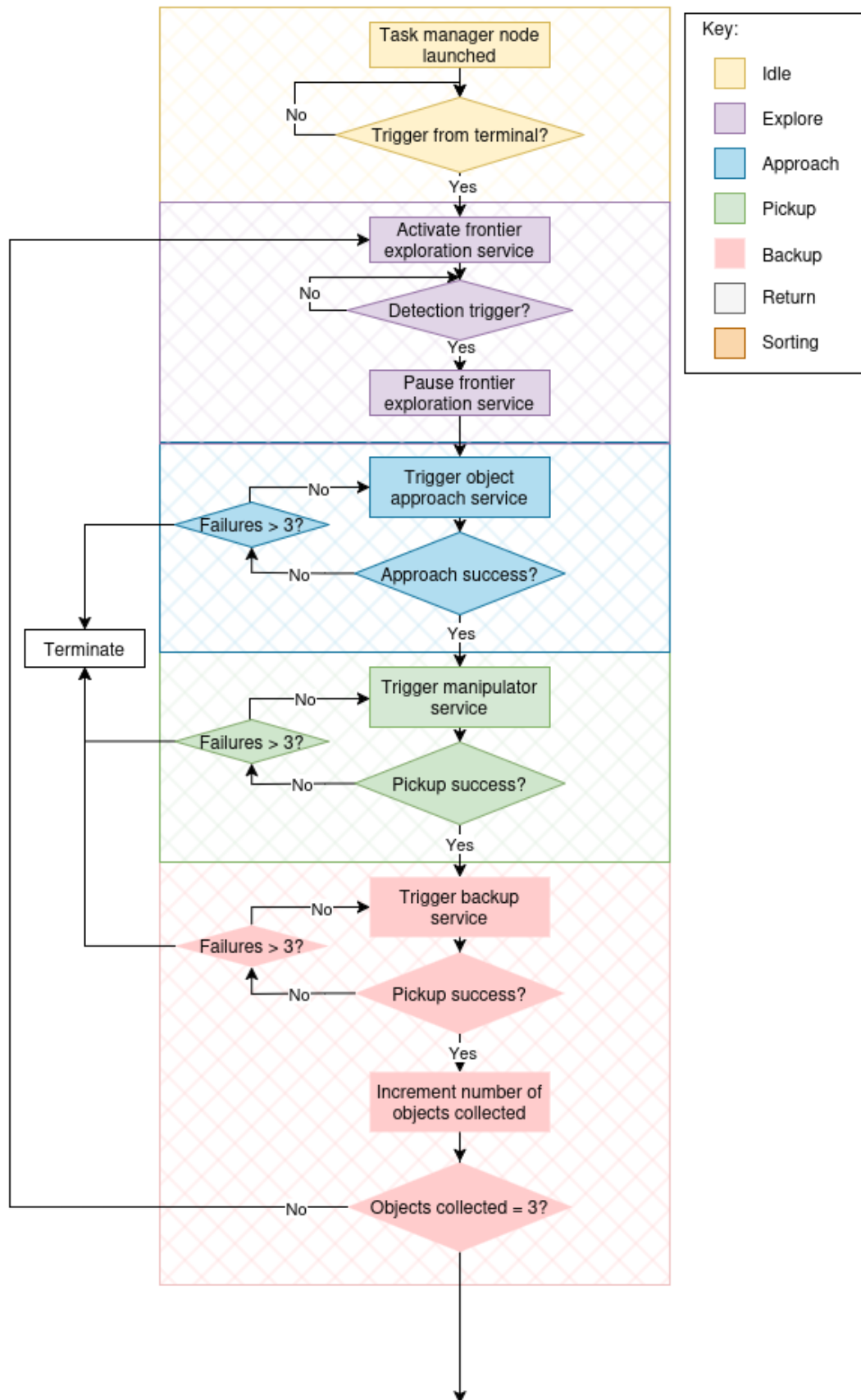


Fig. 16. State machine flow diagram (Part 1).

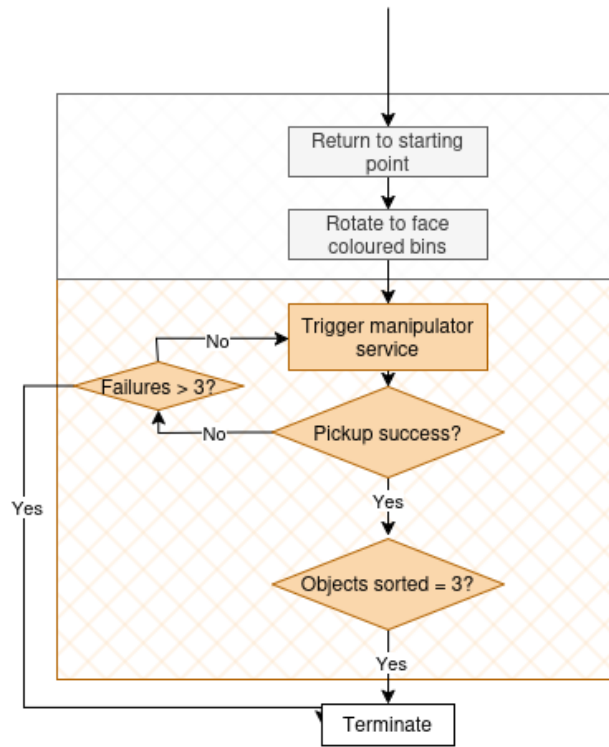


Fig. 17. State machine flow diagram (Part 2).

Initially, after launching all the necessary nodes, the state machine waits in the idle state to receive a trigger service request on the topic `/trigger_rover` from the command terminal. This effectively starts the mission and the state machine switches to the explore state. To activate the frontier exploration node, a True boolean service request is sent on the topic `/explore_status`. The robot begins to navigate the search area autonomously by visiting frontiers, boundaries between explored and unexplored regions of the map. While exploring the map, the state machine waits for a boolean object detection flag from the object perception node on the topic `/detected_flag`. When this message is received, the state machine sends a False boolean service request to the frontier exploration node to pause it, cancelling any active nav2 goals in the process. The state machine then switches to the approach state.

In the approach state, the approach object node is activated using a trigger service request on the topic `/approach_object`. This guides the rover to the location where the object was detected and aligns it so the object is centred in the camera's frame at a distance of 20 cm. Once a True boolean message is received on the topic `/approach_success`, the state machine switches to the pickup state. If the approach process fails three times, the mission is terminated.

In the pickup state, the manipulator service is activated using a trigger request on the topic `/trigger_arm`. The task manager node also publishes a point stamped message containing the storage slot coordinate that the manipulator should place the object into. The manipulator node uses the coordinate of the object published by the object perception node and the coordinate of the storage slot to pickup and store the target object. Once a True boolean message is received on the topic `/manipulator_success`, the state machine switches to the backup state. If the pickup process fails three times, the mission is terminated.

The backup state is required to take the rover out of the obstacle's inflation zone so that nav2 can work again. The backup node is activated using a trigger service request on the topic `/backup`. Once a True boolean message is received on the topic `/backup_success`, the state machine switches to the explore state if less than three objects have been collected, otherwise it switches to the return state. If the backup process fails three times, the mission is terminated.

In the return state, the robot is guided back to the starting point which is effectively (0,0) on the map. Then, it is rotated to face the coloured bins which are also placed near the starting point. After this, the rover switches to sorting state.

In the sorting state, the manipulator service is triggered, it picks the object from the storage slot coordinate and drops it into the coloured bin. This is repeated until all the objects are sorted into their matching bins, after which the mission is terminated.

Figure 18 shows the RQT graph for the task manager node, summarising the topics it is subscribed to.

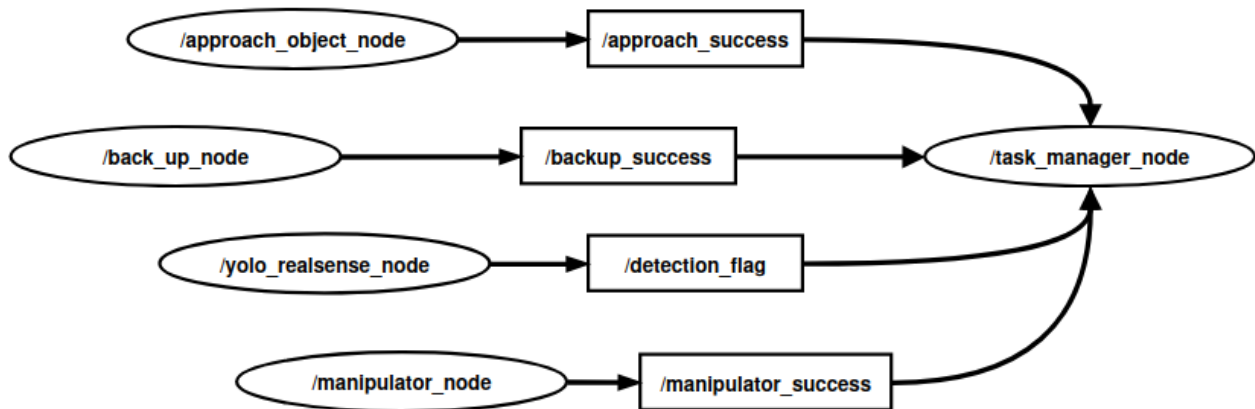


Fig. 18. Frontier exploration node flow diagram.

4.4.3 Scan Filtering

As standoff spacers are used to mount the rover's upper platform, the LiDAR was detecting them as obstacles in its scanning plane. A scan filtering node was therefore introduced to remove these false detections. The RQT graph in Figure 19 shows how the lidar's node publishes `/scan`.

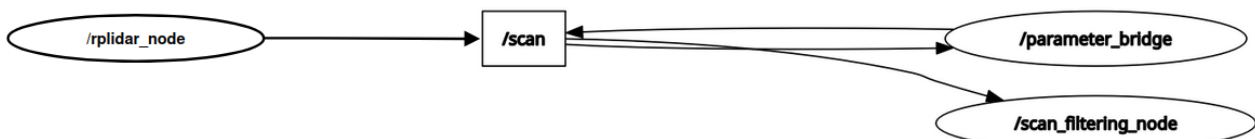


Fig. 19. RQT graph for lidar node.

The scan filtering node subscribes to this topic and removes any detections within a 20 cm radius of the LiDAR, preventing the robot from detecting its own structure as obstacles. Then it publishes a new topic `/filtered_scan` which is used by other nodes such as the slam toolbox. This is shown clearly in Figure 20.

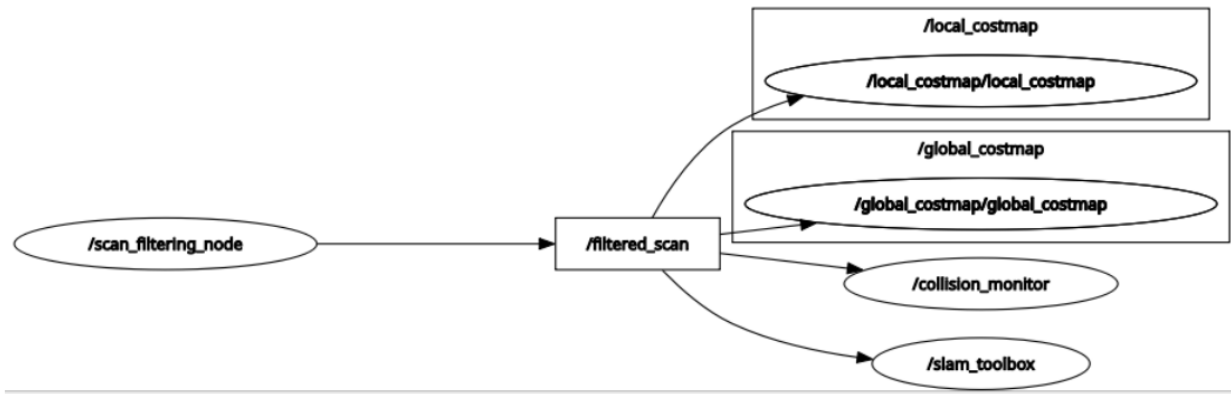


Fig. 20. RQT graph for the scan filtering node.

4.4.4 Frontier Exploration

Figure 21 shows the flow diagram of the frontier exploration node which acts as a service server. After being launched, it waits for a boolean trigger request on the topic `/explore_status`. Once it receives this request, if the request contains a `True` boolean, the explorer is activated, otherwise it is paused. When the explorer is active, it uses `/map` and `/global_costmap` to find accessible frontier cells. If a suitable frontier cell is found, it calls the Nav2 action server with this frontier cell as a goal. Otherwise, it picks a free cell as a goal. If Nav2 successfully completes the action, the process is repeated if the explorer is still active. If Nav2 fails, another frontier or free cell is picked. At any point, if the node receives a `False` boolean service request, the Nav2 action is cancelled safely.

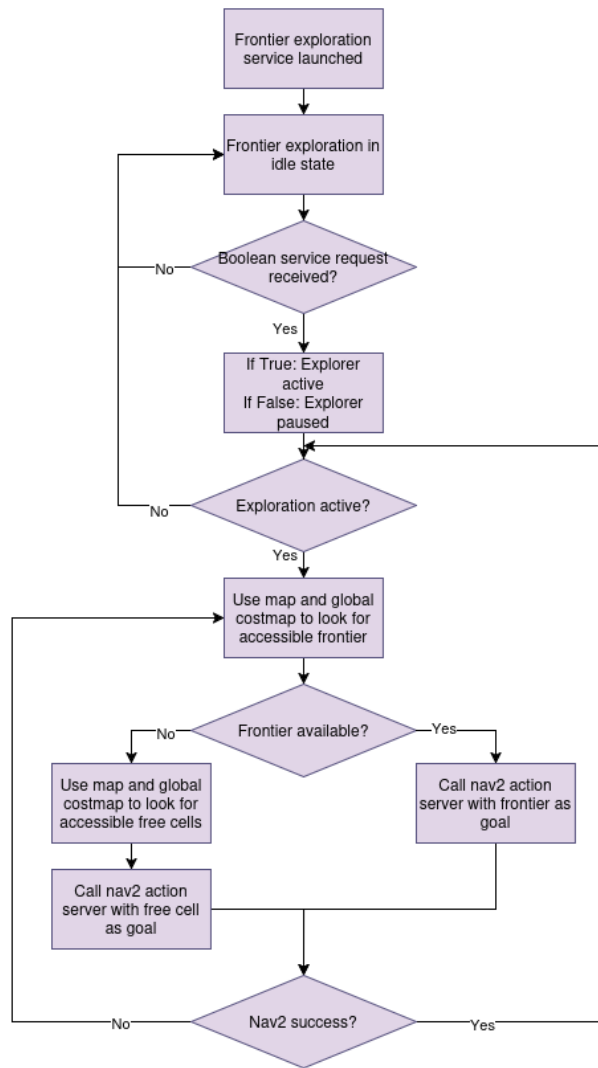


Fig. 21. Frontier exploration node flow diagram.

Figure 22 shows the RQT graph for the frontier exploration node.

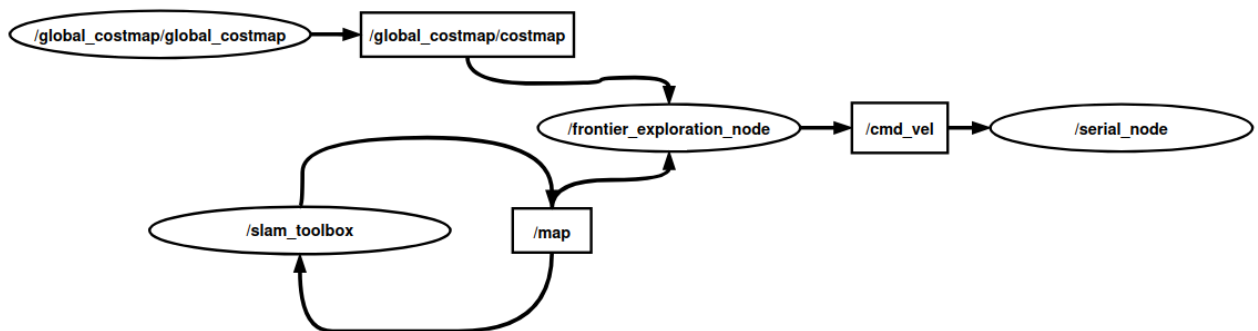


Fig. 22. RQT graph of frontier exploration node.

This node subscribes to `/global_costmap` and `/map`, it also publishes `/cmd_vel` to move the rover through Nav2.

4.4.5 Approaching Object

The approach object node is a service server that guides the robot to an optimal approach point near a target object. Figure 23 shows a diagram of the rover approaching the object. It can be seen that both the box upon which the target object is placed and the rover can be represented as circles. The circle of points in red represents all the places where the rover can safely approach the object without colliding into it. The radius of this circle would simply be the sum of the rover and box's radii and the centre would be the known target object coordinate published by the object perception node.

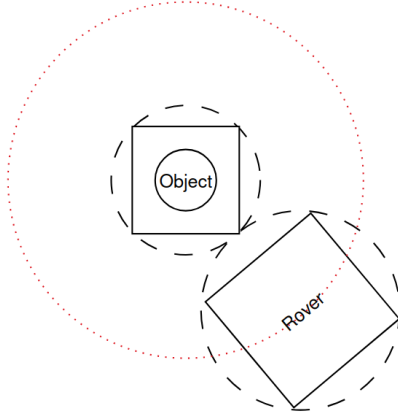


Fig. 23. Diagram of rover approaching object.

Once this circle of candidate points are calculated, the approach object node uses `/global_costmap` to eliminate inaccessible candidates that are too close to obstacles. Then, out of the remaining points, the one with the closest euclidean distance to the current location of the robot is chosen. This point is sent as a Nav2 action goal.

Figure 24 shows the RQT graph for the approach object node.

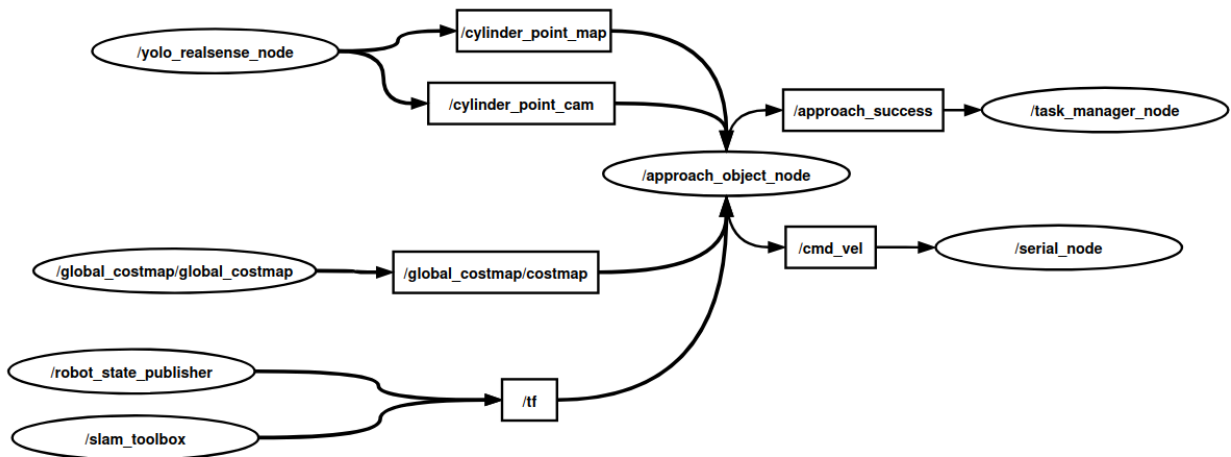


Fig. 24. RQT graph of approach object node.

4.4.6 Manipulator

The manipulator receives cylinder position and colour information from the `/yolo_realsense_node`. This data is continuously published by the camera node running on the Intel NUC via the ROS 2 topic `/cylinder_target`. The manipulator operates a dedicated ROS 2 node that subscribes to this topic and continuously listens for incoming cylinder data.

Upon receiving valid cylinder information, the manipulator waits for a trigger signal from the state machine before initiating the pick-and-place operation. Once triggered, it uses the received cylinder position to command the MyCobot manipulator through the PyMyCobot interface, which communicates directly with the robot hardware via its API.

The manipulation process involves solving inverse kinematics (IK) and executing a trajectory planning algorithm to perform the pick-and-place task. The cylinder data is transmitted using a custom ROS 2 message composed of two standard message types: `Point` and `String`. The `Point` message represents the 3D position of the cylinder, while the `String` message specifies its colour.

A direct mapping between cylinder colour and storage location is implemented within the control algorithm. Based on the detected colour, the manipulator selects the corresponding predefined slot on the rover's storage bin and places the cylinder accordingly. This ensures consistent and deterministic sorting of cylinders during operation.

Figure 25 shows the RQT graph for the manipulator node.

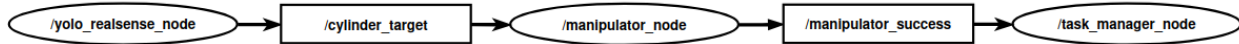


Fig. 25. RQT graph for the manipulator node.

4.4.7 Object Perception

The camera perception node uses an Ultralytics YOLO object detection network with a custom-trained model (`best.pt`) to detect the required coloured target objects. An Intel RealSense D435i provides synchronised RGB and depth streams, with depth aligned to the colour image so that each 2D detection can be extended to 3D target localisation. Depth is estimated from a local region around each detected object and converted into a 3D point in the camera frame using the RealSense intrinsics. To improve stability, the system applies IoU-based tracking and exponential moving average (EMA) smoothing.

The detected 3D point is then transformed through the TF tree from `camera_frame` to `arm_base_link`. The final result is published through the ROS2 topic `/cylinder_target`, which contains both the detected object class and its transformed 3D coordinates in a single JSON-formatted message for the manipulator subsystem. Overall, this pipeline provides reliable coloured object recognition and target localisation for downstream grasping and manipulation tasks.

Figure 26 shows the RQT graph for the yolo realsense node.

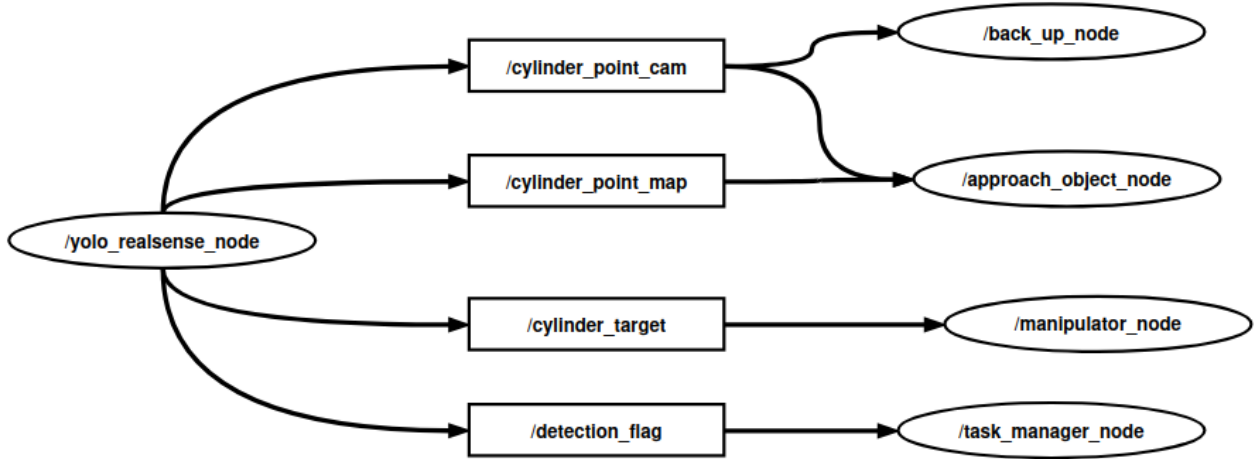


Fig. 26. RQT graph for the yolo realsense node.

4.4.8 Team GitHub Repository

The team's design documents and code can be found at: https://github.com/krishnanair2025/Team9_repository.git. This repository integrates all sub-system code and enables better collaboration between sub-teams during continued development of the rover.

4.5 Analysis: Sub-Team Progress and Implementation Strategy

4.5.1 Navigation and Mapping

Thus far, most of the work related to navigation and mapping has been carried out in simulations of the rover in Gazebo. The custom gazebo world shown in Figure 27 with a 6m x 4m rectangular boundary marking the search area and randomly located shapes marking the obstacles has been used to develop the autonomous navigation and mapping capabilities of the rover.

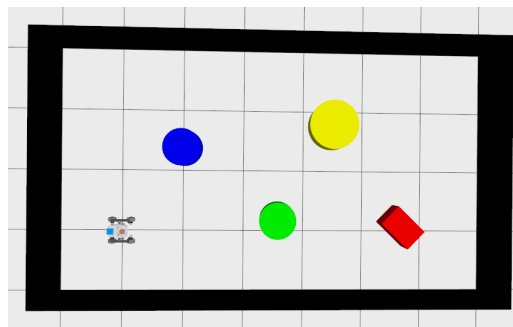


Fig. 27. Custom gazebo world to model the search environment of the robot.

Using the SLAM toolbox, the robot can map its environment using scanning data from its LiDAR and localise itself within the search area using odometry data from the rover's wheel encoders. Figure 28 shows the rover in the process of mapping its environment. This allows the robot to satisfy FR3 and the corresponding PR3.1 in simulation but further tests will be conducted with the physical rover and the specific LiDAR model which will be used on the robot system. Further fine tuning might be required in the SLAM configurations to ensure that the mapping accuracy meets PR3.1. The SLAM toolbox is also responsible for accurate localisation (FR4/PR4.1), in the Gazebo simulation, it has been proven that the robot can update its position as the TF frames move in sync with the simulated rover in Gazebo but the accuracy and magnitude of localisation drift will be measured when conducting physical tests with the LiDAR and rover.

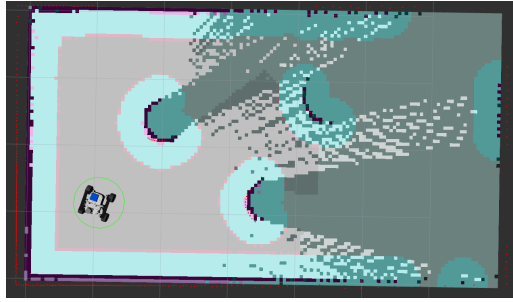


Fig. 28. Simulated robot in the process of mapping its environment in Gazebo.

The Nav2 stack is responsible for navigation, collision avoidance and speed control during the mission. Implementing this in addition to the SLAM toolbox resulted in the rover being able to plan paths to target coordinates within the search area and controlling the robot to travel to that location while avoiding obstacles and actively mapping new areas of the map. In addition to this, a simple frontier exploration node was created to send goals to the Nav2 stack, allowing the rover to explore the map without manual goal input. This resulted in the rover being able to meet PR1.1 and PR1.3 in simulation as it could navigate and map the entire 6m x 4m area without requiring any teleoperation commands. The frontier exploration node will be further developed by improving its awareness of inflation zones on the costmap when sending goals.

PR1.2 was met by setting the Nav2 controller server's `vx_max` parameter to be 0.25 m/s which aligns with the risk assessment's maximum speed limit of 250 mm/s. Nav2 can avoid collision using inflation layers in cost maps and the robot's radius. To ensure that the robot does not get closer than 50mm to any obstacle or wall, the robot's footprint was enlarged to include this buffer zone and the inflation radius was also set strategically. The rover has dimensions of 425mm x 448mm, a 50mm clearance was added to each side resulting in the footprint of the robot being defined as a circle with a radius of 0.37m. In addition to this, the inflation zone in both local and global costmaps was set to 0.37m. This resulted in the rover maintaining a 50mm clearance from all obstacles throughout its exploration of the 6m x 4m area. This means that the robot satisfies PR2.1 in simulation.

Having proven that navigation and mapping is operational in a simulated environment, the next step was to implement this on the physical rover. Firstly, it was necessary to implement the mapping feature by running SLAM toolbox while using the in-built keyboard teleop package to move the rover. It was observed that the rover produced an accurate map of the test area while continuously localising itself within the generated map, further proving its ability to satisfy FR3, PR3.1, FR4 and PR4.1. Following this, it was planned to implement autonomous exploration by implementing the frontier exploration node on the physical rover. A test was conducted within a search area created using foam board walls as shown in Figure 29, it was observed that the rover autonomously explored the search area.

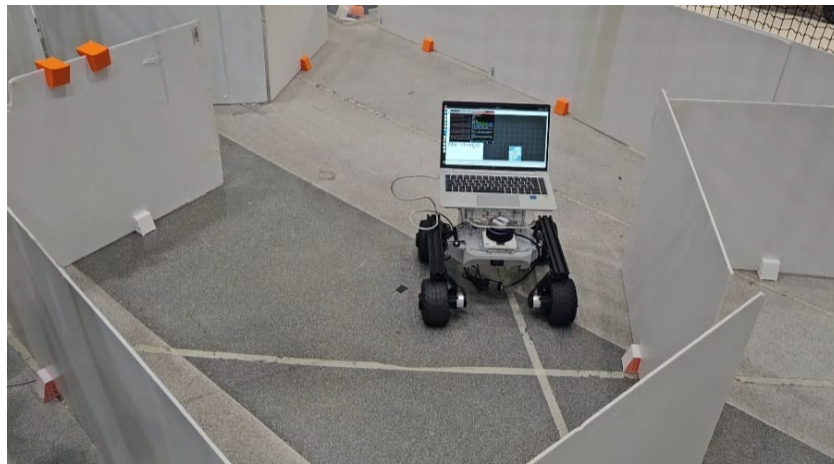


Fig. 29. Robot autonomously navigating test area.

The next steps are to integrate with the vision sub-team to develop the feature of exploring till a target object is detected, approached, picked and stored. This must be repeated without any failures until all three objects are collected.

Table 2. Navigation and mapping requirements verification

Requirements	Design Solutions	Verification Method Used
FR1 - The robot shall fully autonomously navigate the search area containing obstacles.	The frontier exploration node enables the rover to autonomously explore the search area by continuously moving towards boundaries between explored and unexplored regions.	It was observed that the rover autonomously explores the search area in both Gazebo simulations and physical tests in the labs.
PR1.1 - The robot shall be able to navigate a 6m x 4m area.	The rover is capable of navigating areas of various sizes, the only limitation being the battery capacity.	In a simulated Gazebo environment, under the speed restrictions of lab risk assessments, the rover was capable of exploring a 6 x 4m search area.
PR1.2 - The robot shall not exceed a maximum speed of 250 mm/s.	Using the Nav2 parameter file, the maximum speed of the rover has been set to be 250 mm/s. This sets a hard limit on how fast the rover can move while executing the mission.	In both practical tests and gazebo simulations, monitoring the command velocity topic has shown that the velocity never exceeds 250 mm/s, thus meeting the criteria.
PR1.3 - The robot shall navigate the search area without requiring teleoperation or physical intervention.	Frontier exploration automatically sends nav2 goals to move the rover towards unexplored regions of the map, this means that the rover is capable of exploring the search area without manual control through teleoperation.	The rover was launched once at the start of each trial and completed exploration without any teleoperation commands or physical intervention. This was demonstrated in both Gazebo simulation environment and physical testing.
FR2 - The robot shall avoid collisions with walls or obstacles	Due to careful Nav2 parameter tuning, the LiDAR and accurate SLAM, the rover successfully avoids obstacles in the search area.	In both simulation and physical tests, the rover navigated around obstacles and walls without any collisions, confirmed through observation and navigation logs.
PR2.1 - The robot shall maintain a clearance of at least 50mm from walls and obstacles during the mission.	Due to in-built collision avoidance algorithms in the Nav2 stack as well as carefully chosen inflation zone parameters in the Nav2 parameter file, the rover never gets closer than 50mm to the walls or obstacles.	Clearance was monitored in RViz using costmaps and LiDAR data, showing the rover consistently maintained more than 50mm distance from obstacles.
PR3 - The robot shall detect and map obstacles within the search area	The LiDAR has been set up and mounted on the rover's payload sled in such a way that it has maximum coverage of its surroundings. This means that the rover has clear visibility of obstacles, allowing it to detect and map them through SLAM.	The generated occupancy grid map was compared with the known environment layout, confirming correct detection and mapping of obstacles.

Requirements	Design Solutions	Verification Method Used
PR3.1 - The map generated shall match the physical layout with a tolerance of $\pm 5\text{cm}$	To achieve this level of tolerance, the mapping resolution has been set to be $\pm 5\text{cm}$, matching the requirement of this project. This allows the rover to accurately map its surroundings.	Measured distances in the real environment were compared to the generated map, showing deviations within $\pm 5\text{cm}$.
FR4 - The robot shall estimate and update its position within the map during navigation.	LiDAR-based SLAM combined with AMCL localisation continuously estimates and updates the robot pose.	The robot pose in RViz was observed to update continuously and remain consistent with its actual movement.
PR4.1 - The estimated position of the robot shall be within $\pm 0.05\text{m}$ of its actual location in the search area.	Accurate wheel odometry combined with LiDAR scan matching ensures pose estimation error is minimal.	It was observed during demonstrations of autonomous navigation that the rover's position was correctly being updated in RViz along with its physical movement.

4.5.2 Manipulator

The myCobot 280 PI manipulator, shown in Figure 15, is pre-installed with ros2-jazzy distribution and the pymycobot library from the official Elephant Robotics Github website. It also features built-in, hardware level kinematic solver to perform general trajectory following, and internal control system to achieve a stable joint space and task space position, velocity and acceleration control. Overall, the manipulator features a comprehensive, ready-to-code software and hardware that allows any more complex functionalities to be customly built from scratch without needing external libraries or hardware.

Due to the nature of the experimental task and the workspace required for the manipulation operations, the final system implementation does not utilise the MoveIt2 motion planning framework. Instead, the manipulator is controlled directly through the myCobot Python library, which provides low-level joint control and simple trajectory execution through direct hardware API connection. This decision was made to reduce software complexity, minimise computational overhead, and improve reliability during the short mission duration. For the specific task requirements of the project, complex motion planning was not necessary, as the robot only needs to pick up cylindrical objects from the environment and place them into predefined storage locations on the rover. Therefore, the team implemented a custom pick-and-place pipeline tailored specifically for this task.

4.5.3 Achieved Requirements & their Analysis

The manipulator can be quickly deployed in any real-life applications to perform tasks such as object grasping [FR6, PR6.1, PR6.2], placing [PR6.3, FR7] and moving through trajectories [FR8]. The ROS2 package are fully integratable. Meaning it's easy to add any custom functions or nodes with additional functionalities for perform more complex or varied tasks.

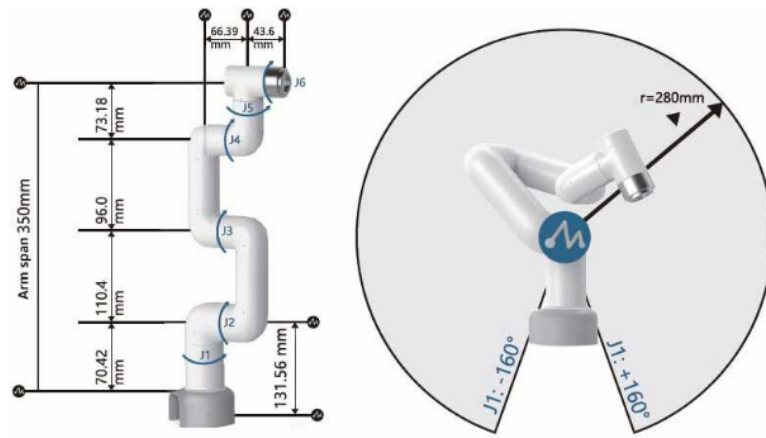


Fig. 30. The myCobot 280 Raspberry Pi manipulator.

The table below shows a detailed analysis of how our robotic manipulation design fulfils the aforementioned requirements, and the tests we have performed to verify that requirements of object manipulation have been achieved.

Table 3. Object manipulation requirements verification.

Requirements	Design Solution	Verification Method Used
FR6 - Successful grasping of objects	By using our custom path planning algorithm, the robot will: 1. First return to a neutral position to avoid hitting obstacles. 2. Rotate so that it's facing directly at the target object. 3. Perform IK operation such that the gripper is positioned slightly above the target object. 4. The gripper is slower gradually using linearly interpolated trajectory to ensure a direct grasp. 5. The manipulator then lift the object up by rotating it's base joint before moving to avoid knocking objects off.	• Demonstration: 1. Placed the target object within the manipulator's workspace 2. Ran object recognition program and manipulator publisher node How success was verified: • Inspection: Manipulator received the target object's coordinates, displayed it in terminal, generated and followed a planned path, grasp target object, lifted it up and returned to a neutral configuration.
PR6.1 - The robot shall pick up cylinders of 26mm diameter and 60mm height, weighing 20g.	The grasp-from-above strategy ensures the gripper is parallel to the cylinder's top plane while centered at its radius. This allows the cylinder object to be picked up at and z-axis orientation, which increases task flexibility. The gripper has a minimum clamp width of 20 mm and a max clamping weight of 150g. This is more than capable of firmly gripping the target object without slipping. The depth camera is integrated with the manipulator to provide information so that the manipulator can distinguish between cylinders and non-cylinders.	• Demonstration: Placed the object in front of manipulator and verified pick-up 10 times. The manipulator's subscriber node can be programmed to run repeatedly in the event that the target grasping location does not change. How success was verified: • Inspection: All objects remained secure in gripper during picking and all objects inside the correct target container at the end of placing.
PR6.2 - The robot shall pick up 3 target objects during the mission.	The manipulator's node is subscribed to the topic <code>/yolo_realsense_node</code> . It can be programmed to perform pick-and-place operation only once every time it receives a target coordinate from the Intel NUC via that topic. During exploration, the depth camera scans the environment and returns the location of the cylinder objects one at a time.	• Demonstration: Ran mission with 3 randomly placed objects and verified the collection. How success was verified: All 3 objects collected correctly in all trials.

Requirements	Design Solution	Verification Method Used
PR6.3 - The robot shall put all three cylindrical objects into onboard storage.	The coordinates of the 3 bin holes are stored in the manipulator's Raspberry Pi processor. The manipulator is programmed to lower the cylinder object into the hole before relasing the gripper to ensure the object is dropped securely inside the container hole.	• Demonstration: Ran mission with 3 randomly placed objects and verified the collection. The manipulator's subscriber node program performed pick-and-place operation only once every time it received a target coordinate while the trigger signal being present. How success was verified: All 3 objects collected correctly in all trials.
FR7.0 - Successful placing of aquired objects in target containers	We have built a custom ros2 message that encodes: 1. The shape and color of the object. 2. The location of the target object. The label of the object is used to update the manipulator's action decision of whether is picking the cylinder or placing the cylinder into the colored bins. Once the nature of the task is determined, the manipulator will carry out pick-and-place operation as before.	• Demonstration: 1. Sent the target message including the target label and location (can be the on-board storage bin or an arbitrary location in the environment). 2. Manipulator moved towards the location slightly above the target. 3. The gripper opened in the correct target position and released the object. How success was verified: Target object was inside the target container.
FR8.0 - Successful sorting of objects with matching color	The depth camera returns the coordinate and labels of the target object. These contains information of both the location of the target object and it's shape/color. 2. We assigned 3 fixed coordinates which corresponds to the storage bin locations for each color. The manipulator can be commanded to pick/place the correct colored objects from/into their corresponding bin location.	• Demonstration: 1. Placed 3 cylinders of different colors at arbitrary locations on the floor. 2. Ran navigation algorithm and depth camera's object detection algorithm. 3. The cylinder's label and coordinates were sent to the manipulator. 4. The manipulator performed pick-and-place for each cylinder. How success was verified: The object was located at the bin holes that corresponds to the cylinder's colour.

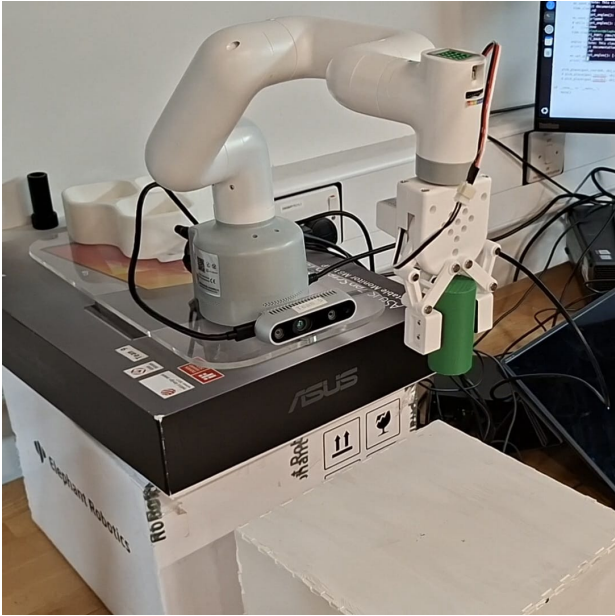


Fig. 31. Integration of perception and manipulation in a pick-and-place task. The Intel RealSense depth camera provides visual and spatial information about the target object, while the MyCobot 280 Pi manipulator uses this data to accurately locate, grasp, and move the object. This demonstrates the coordinated operation between the vision system and the manipulator to achieve autonomous pick-and-place functionality.

The perception system uses the RGB-D camera to detect the coloured cylinders. Once an object is detected, the camera pipeline extracts the centroid of the detected cylinder in the image frame. This centroid is then combined with depth information to estimate the 3D position of the object relative to the robot frame. A coordinate transformation is then applied to convert this pose into the manipulator base frame, allowing the robot arm to reach the target location.

Using this position estimate, the manipulator executes a predefined grasp sequence implemented using the myCobot control interface. The arm moves to a pre-grasp pose above the detected cylinder, lowers to the grasp position, closes the gripper to secure the object, and lifts the object from the environment. After a successful grasp, the robot places the object into a designated storage location on the rover, which is determined by the colour of the detected cylinder. Each colour corresponds to a predefined placement pose within the onboard storage bin. The manipulator then executes a placement trajectory to deposit the object into the correct compartment before returning to a neutral pose to continue the mission. This simplified manipulation pipeline provides a task-specific and computationally efficient solution while maintaining reliable object retrieval and placement performance for the experimental scenario.

4.5.4 Object Perception

The camera perception module acts as the main sensory component of the robot, using a custom-trained YOLOv8 model to detect red, yellow, and green cylinders and bins. An Intel RealSense D435i depth camera provides aligned RGB and depth streams, allowing 2D detections to be extended into accurate 3D localisation, fulfilling functional requirement FR5. A custom dataset was captured under varied distances, angles, and lighting conditions to ensure robust detection. Images were annotated on Roboflow with consistent class labels, then split into training, validation, and test sets and exported in YOLOv8 format. The YOLOv8s model was trained using transfer learning with suitable hyperparameters and early stopping to avoid overfitting. After approximately nine hours of training on a CPU, the final model achieved high precision and strong overall detection performance, with the best weights saved for deployment.

During real-time inference within a ROS 2 node, the model processes camera input and outputs bounding boxes and class labels. Low-confidence detections are filtered to improve reliability. Depth information from the camera is used to calculate 3D coordinates for each detected object in the camera frame. To reduce jitter and improve stability across frames, IoU-based tracking and EMA smoothing

are applied to position estimates.

The 3D coordinates are transformed via the ROS 2 TF system from the camera frame to the manipulator base frame, aligning target positions with the robot's kinematic structure. Results including object class and spatial coordinates are published to a ROS 2 topic for use by the grasping and control system. This pipeline ensures accurate cylinder detection as required by PR5.1 and maintains localisation accuracy within $\pm 10\text{mm}$ to meet PR5.2, enabling consistent and dependable autonomous object detection and positioning.

This table summarizes the requirements met by the computer vision system:

Table 4. Object perception requirements verification summary

Requirements	Design Solution	Verification Method Used
FR5 - The robot shall detect the location of target objects PR5.2 - The robot shall provide the location of the target object with an accuracy of $\pm 10\text{mm}$	The yolo model has a built in depth computation algorithm to return the depth of any recognized objects	• Test: 1. Place all target objects in front of the camera. 2. Displays the output label of each objects on screen. How success was verified: The labels of each object matched with their respective true colours.
PR5.1 - The robot shall detect red, yellow and green cylinders.	We trained our models specifically on our target object (the colored cylinders). We have taken images of our cylinders at various distances and angles to get a comprehensive dataset. The yolo model can converge faster and provides a robust recognition performance.	• Test: 1. Placed target object at a fixed distance from the camera. 2. Printed the output of the yolo model on terminal. 3. Computed the error between the detected location and the object's actual location. How success was verified: The error was less than a pre-defined threshold.

4.5.5 Power Requirements

The design solution of using 2 additional power banks ensures that the robot can reliably perform perception, mapping, and manipulation planning tasks throughout the mission while maintaining sufficient energy capacity for the entire operational period. Providing a dedicated onboard power source for the compute subsystem also contributes to satisfying FR9, PR9.1 and PR9.2 by ensuring reliable untethered operation.

The separation of power systems also improves reliability during manipulation tasks and guarantees sufficient energy capacity for the duration of the mission. As with the compute subsystem, powering the manipulator from an onboard rechargeable power bank ensures that the robot operates fully untethered, satisfying FR9, PR9.1 and PR9.2.

4.6 Project Plan

The Gantt chart in Figure 32 outlines the Leo Rover project's Semester 2 work plan, with relevant context from Semester 1, covering five core technical modules and key milestones as well as deliverables:

- **Hardware & Manufacture:** The team undertook the design and fabrication of the payload sled and mounts, a foundational hardware component that was completed in Semester 1.

- **Computer Vision (CV):** The team carried out YOLO model training, dataset preparation, precision refinement for failure cases, depth-to-3D coordinate transformation, and integration with the TF tree—all essential for object detection and spatial perception.
- **Navigation (NAV):** The team completed simulation setup, basic SLAM testing, Nav2 stack configuration and parameter tuning, developed an autonomous frontier exploration node, and designed and tested a task management node to enable autonomous navigation capabilities.
- **Manipulation (MANI):** The team performed mechanical arm assembly, basic driver communication, speed and acceleration control testing, trajectory planning, grasp logic development, and calibrated the arm with the camera frame, laying the groundwork for physical interaction with objects.
- **System Integration:** The team collaborated to develop a central Task Manager state machine, integrated the Computer Vision and Manipulation modules, and conducted full system integration and timed trials to ensure end-to-end functionality.

Leo Rover Project Plan

Semester 2 Schedule (with Semester 1 Context)



Fig. 32. Gantt chart with work packages.

Deliverables

- Explore node package
- Task manager node package
- Physical payload assembly
- Object detection node package
- Manipulator control nodes

Appendix: Updated design requirements document + workplace charter

Fig. 33. Deliverables

References

- [1] J. R. Sánchez-Ibáñez, C. J. Pérez-del-Pulgar, and A. García-Cerezo, "Path planning for autonomous mobile robots: A review," *Sensors*, vol. 21, no. 23, 2021, ISSN: 1424-8220. DOI: 10.3390/s21237898. [Online]. Available: <https://www.mdpi.com/1424-8220/21/23/7898> (cited on p. 6).
- [2] B. Yildiz et al., "Cnn based sensor fusion method for real-time autonomous robotics systems," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 30, no. 1, pp. 79–93, 2022 (cited on p. 6).
- [3] K. Charalampous, I. Kostavelis, and A. Gasteratos, "Thorough robot navigation based on svm local planning," *Robotics and Autonomous Systems*, vol. 70, pp. 166–180, 2015 (cited on p. 6).
- [4] A. Elashry and M. Yasser, "A formally verified framework for indoor positioning with wi-fi fingerprinting, bov, and orb," *BoVW, and ORB*, (cited on p. 6).
- [5] L. Mochurad, Y. Hladun, and R. Tkachenko, "An obstacle-finding approach for autonomous mobile robots using 2d lidar data," *Big Data and Cognitive Computing*, vol. 7, no. 1, p. 43, 2023 (cited on p. 6).
- [6] J. Sun et al., *Autonomous navigation system of indoor mobile robots using 2d lidar. mathematics* 2023, 11, 1455 (cited on p. 6).
- [7] fictionlab, "Leo rover 1.9," 2025, Available at: <https://fictionlab.pl/shop/leo-rover-1-9/> (cited on p. 11).
- [8] T. A. Association, "Aluminum's sustainability," 2025, Available at: <https://www.aluminum.org/sustainability> (cited on p. 11).
- [9] futerro, "Pla, a durable and sustainable polymer for a greener future," 2025, Available at: <https://www.futerro.com/wp-content/uploads/2025/09/WHITEPAPER-FUTERRO-WEB-EN.pdf> (cited on p. 11).
- [10] BioPak, "What is pla plastic?," 2025, Available at: <https://www.biopak.com/uk/resources/what-is-pla> (cited on p. 11).
- [11] cutmy, "Clear acrylic sheets," 2025, Available at: <https://www.cutmy.co.uk/plastic/acrylic-perspex-sheets/> (cited on p. 11).
- [12] CORDIS, "Sustainable rubber for the environment.," 2025, Available at: <https://doi.org/10.3030/190132096> (cited on p. 11).

- [13] M. R. Company, "A guide to rubber recycling.," 2025, Available at: <https://www.martins-rubber.co.uk/blog/guide-to-rubber-recycling/> (cited on p. 11).
- [14] E. Robotics, "Adaptive gripper," 2025, Available at: https://docs.elephantrobotics.com/docs/mycobot_280_m5_en/4-SupportAndService/Accessories/AdaptiveGripper.html (cited on pp. 12, 28).
- [15] SULAPAC, "Replacing abs plastic sustainably," 2025, Available at: <https://www.sulapac.com/blog/replacing-abs-plastic-sustainably/> (cited on p. 12).
- [16] BuhlerMotor, "Sustainability," 2025, Available at: <https://www.buehlermotor.com/sustainability.html> (cited on p. 12).
- [17] I. Batteries, "Samsung 18650 batteries | high-performance lithium-ion cells," 2025, Available at: <https://www.allbatteries.co.uk/lithium-ion-battery-samsung-inr18650-35e-3-6v-3-5ah-ft-acl9311b10.html> (cited on p. 12).
- [18] ElephantRobotics, "Mycobot 280 pi," 2024, Available at: <https://www.elephantrobotics.com/en/mycobot-280-pi-specifications-en/> (cited on p. 28).
- [19] fictionlab, "Leo rover specification," 2024, Available at: <https://docs.fictionlab.pl/leo-rover/documentation/specification> (cited on p. 29).

5 Appendix

5.1 Appendix A: Design Requirements

5.1.1 Problem Statement

The objective of this project is to design and develop a fully autonomous rover capable of navigating a rectangular search area with randomly placed obstacles to collect 3 coloured objects and return them to matching coloured bins at the rover's starting point within a 20-minute operating time.

5.1.2 Concept of Operations (ConOps)

Figure 34 shows a diagram of the environment that the rover is required to operate in, based on the problem statement and stakeholder responses.

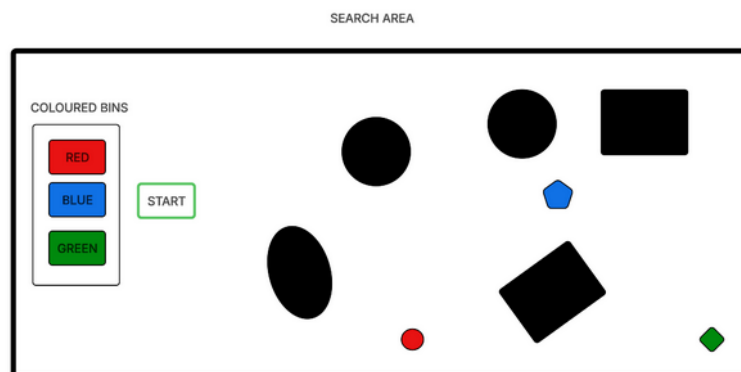


Fig. 34. Diagram of the expected environment.

There is a rectangular search area of approximately 6m x 4m marked by short walls which the robot must operate within. The black shapes represent obstacles and the coloured shapes represent the target objects, both of which are randomly placed inside the search area. On one side of the search area, there are colour-coded bins and a starting point for the robot.

After preparing the robot for the mission, it will be placed at the starting point. During the 20 minute mission time, the robot will autonomously navigate the search area while its vision system monitors the environment for coloured objects. If an object is detected, the robot will approach the detected object and the manipulator will pick and place the object into the onboard storage bin. The robot continues doing this until all three objects are collected at which point it returns to the starting point and the manipulator picks each object from the onboard storage bin and places it in a matching coloured

bin. Once all three objects are sorted the mission is complete and the robot can be packed up. The flowchart in Figure 35 shows the ConOps in the form of a flowchart.

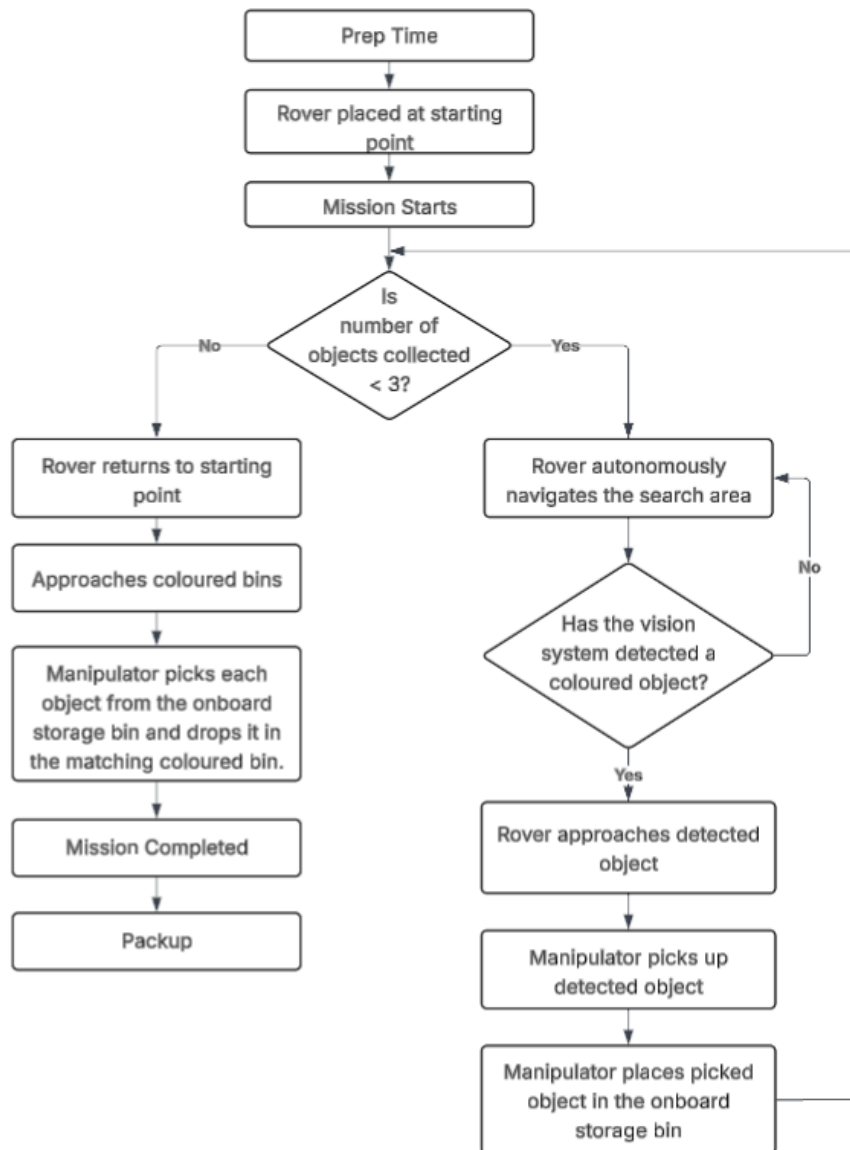


Fig. 35. Flowchart of the ConOps.

5.1.3 Functional and Performance Requirements

Table 5. Functional and Performance Requirements with Justifications

Req. ID	Description	Justification
FR1	The robot shall fully autonomously navigate the search area containing obstacles.	The robot must navigate the search area to find target objects.
PR1.1	The robot shall be able to navigate a 6m x 4m area.	The search area has dimensions of 6m x 4m.
PR1.2	The robot shall not exceed a maximum speed of 250 mm/s.	The risk assessment limits maximum speed to 250 mm/s.
PR1.3	The robot shall navigate the search area without requiring teleoperation or physical intervention.	No physical intervention allowed unless the robot malfunctions.
FR2	The robot shall avoid collisions with walls or obstacles.	The robot must not hit obstacles or walls.
PR2.1	The robot shall maintain a clearance of at least 50mm from walls and obstacles during the mission.	Leaving a minimum clearance of approximately 50% of the robot's width allows for safe navigation within the search area.
FR3	The robot shall detect and map obstacles within the search area.	The robot does not have a map of the environment; it is required to generate its own during the mission.
PR3.1	The map generated shall match the physical layout with a tolerance of ± 5 cm.	The robot must be capable of making an accurate map of its environment. 5cm is the common mapping resolution used by SLAM.
FR4	The robot shall estimate and update its position within the map during navigation.	The robot must be able to localise itself in its environment for effective navigation and mapping.
PR4.1	The estimated position of the robot shall be within ± 0.05 m of its actual location in the search area.	The robot must localise itself accurately to improve navigation performance. The mapping resolution is 0.05m for SLAM, so this is a reasonable tolerance.
FR5	The robot shall detect the location of target objects.	The robot should be able to detect and locate target objects in the search area.
PR5.1	The robot shall detect red, yellow and green cylinders.	The shapes placed in the search area will be red, yellow and green cylinders.

PR5.2	The robot shall provide the location of the target object with an accuracy of $\pm 10\text{mm}$.	The location of the object must be detected accurately to allow the manipulator to pick it without errors. Since the cylinder has a diameter of 26mm and the fully open position of the gripper is 37mm wide, a tolerance of 10mm still puts the cylinder inside the open gripper.
FR6	The robot shall pick up target objects using its manipulator.	The robot must be able to collect the target object from the search area.
PR6.1	The robot shall pick up cylinders of 26mm diameter and 60mm height, weighing 20g.	The manipulator must be capable of picking up three identical wooden cylinders placed in the search area.
PR6.2	The robot shall pick up 3 target objects during the mission.	The mission is only completed if all three target objects are collected.
PR6.3	The robot shall put all three cylindrical objects that it picks into its onboard storage facility.	The robot must take all three target objects to the starting point.
FR7	The robot shall transport collected objects to the starting point.	The objects must be collected and sorted into bins which are located at the starting point.
PR7.1	The robot shall securely hold 3 objects onboard.	The robot must take all three target objects to the starting point.
FR8	The robot shall sort the target objects into bins with matching colour.	The coloured target objects must be sorted correctly into matching-coloured bins.
PR8.1	The robot shall detect red, green and yellow bins.	The robot must detect red, green and yellow bins to sort the red, green and yellow target objects it collected.
FR9	The robot shall operate, untethered for at least 20 minutes.	The mission duration is 20 minutes and no wired connections to the robot are allowed.
PR9.1	The robot's onboard energy storage system shall support all sub-systems for 30 minutes of operation.	The mission duration is 20 minutes. A safety margin of 10 minutes will improve reliability.
PR9.2	The robot shall not use trailing power or data cables during the entirety of the mission.	The robot must operate untethered.

FR10	The robot shall maintain mechanical stability during navigation and manipulation tasks.	Robot must be mechanically stable to enable safe operation during the mission.
PR10.1	All the components on the robot shall securely remain in place over the duration of the mission.	Peripheral components moving and shifting out of place could negatively affect mapping, vision and manipulation functionalities. Collected objects could also be lost.
FR11	The robot shall complete the entire mission within the allocated 20-minute time limit.	The mission must be completed within 20 minutes.
FR12	The robot shall handle all computational work onboard.	The robot will be less reliant on network connections, avoiding issues with latency.

5.1.4 Requirements Verification Matrix

Table 6. Requirements Verification Matrix

Req. ID	Description	Verification Method	Verification Level	Verification Description	Success Criteria	Rationale
FR1	The robot shall fully autonomously navigate the search area containing obstacles.	Demo*	Subsystem Rover base**, Navigation	Run the robot in the search area and observe whether the rover is capable of fully autonomous navigation.	The robot navigates the entire search area independently.	R1 Navigation is one of the fundamental functions of mobile robots.
PR1.1	The robot shall be able to navigate a 6m x 4m area.	Demo	Subsystem Rover base, Navigation	Observe if the rover can completely cover the 6m x 4m area during autonomous navigation.	The robot travels and maps the entire 6m x 4m area autonomously.	R1.1 In real-life scenarios, mobile robots operate in constrained areas.
PR1.2	The robot shall not exceed a maximum speed of 250 mm/s.	Test + Inspection	Subsystem Rover base	Run the robot in the search area 10 times and observe that the velocity does not exceed maximum limit.	Recorded encoder data shows speed remains below 250 mm/s in all repetitions.	R1.2 Ensures safety and power limit compliance.
PR1.3	The robot shall navigate the search area without requiring teleoperation or physical intervention.	Demo	Subsystem Rover base, Navigation	Run the robot and observe autonomous navigation without physical intervention.	Robot navigates independently without teleoperation.	R1.3 Ensures autonomy as required in industrial applications.

FR2	The robot shall avoid collisions with walls or obstacles.	Demo	Subsystem Rover, Navigation	Operate the robot and verify no collisions occur.	Robot never collides with walls or obstacles.	R2 Obstacle avoidance is required to achieve [FR1, R1].
PR2.1	The robot shall maintain a clearance of at least 50mm from walls and obstacles during the mission.	Demo	Subsystem Navigation, Camera	Mark 50mm clearance and run the robot 10 times to check it never crosses lines.	Rover never crosses tape marking 50mm clearance.	R2.1 Ensures [FR2, R2] are satisfied.
FR3	The robot shall detect and map obstacles within the search area.	Inspection	Subsystem Rover base, Navigation	Run robot and visually confirm live generated map matches surroundings.	Robot correctly maps environment and detects obstacles.	R3 Environment mapping is essential in [FR1, R1].
PR3.1	The map generated shall match the physical layout with a tolerance of $\pm 5\text{cm}$.	Analysis	Subsystem Rover base, Navigation	Measure dimensions and verify map in Rviz matches within tolerance.	Map accuracy $\pm 5\text{cm}$ in 10/10 trials.	R3.1 Ensures [FR3, R3] are satisfied.

FR4	The robot shall estimate and update its position within the map during navigation.	Inspection	Subsystem Rover base, Navigation	Observe that robot location matches Rviz visualization.	TF in Rviz matches real robot movement.	R4 Self-localization is a key SLAM task.
PR4.1	The estimated position of the robot shall be within $\pm 0.05\text{m}$ of its actual location.	Test	Subsystem Rover base, Navigation	Pause exploration every 30s, record position, and compare to actual coordinates.	Estimated position does not drift beyond $\pm 0.05\text{m}$ in all 3 repetitions.	R4.1 Accurate tracking ensures correct navigation.
FR5	The robot shall detect the location of target objects.	Test	Subsystem Rover base, Computer vision, Depth camera	Place cylinders at known positions and verify camera detection 10 times per color.	Camera detects all cylinders correctly each trial.	R5 Ensures objects can be detected for manipulation.
PR5.1	The robot shall detect red, yellow and green cylinders.	Demo	Subsystem Rover base, Computer vision, Depth camera	Place cylinders in 10 positions each and verify detection.	Camera detects all cylinders 10/10 times.	R5.1 Ensures [FR5, R5] are satisfied.
PR5.2	The robot shall provide the location of the target object with $\pm 10\text{mm}$ accuracy.	Test + Inspection	Subsystem Rover base, Computer vision, Depth camera	Place cylinder at known (x,y,z) and verify coordinates 10 times.	Camera outputs coordinates within $\pm 10\text{mm}$ in all 10 repetitions.	R5.2 Provides accurate spatial coordinates for manipulation.

FR6	The robot shall pick up target objects using its manipulator.	Demo + Inspection	Subsystem Rover base, Manipulator	Run pick-up process and verify successful grasp.	Manipulator picks up object without dropping or knocking it.	R5.3 Ensures [FR5, R5] are satisfied.
PR6.1	The robot shall pick up cylinders of 26mm diameter and 60mm height, weighing 20g.	Demo + Inspection	Subsystem Rover base, Manipulator	Place object in front of manipulator and verify pick-up 10 times.	Object remains secure in gripper without being knocked over.	R6.1 Ensures successful grasping.
PR6.2	The robot shall pick up 3 target objects during the mission.	Demo	Subsystem Rover base, Task manager, Manipulator	Run mission with 3 randomly placed objects and verify collection.	All 3 objects collected correctly in all trials.	R6.2 Ensures [FR5, R5] are satisfied.
PR6.3	The robot shall put all three cylindrical objects into onboard storage.	Demo	Subsystem Rover base, Manipulator	Place cylinders and verify correct placement 10 times.	All cylinders stored correctly in each trial.	R6.3 Ensures [FR5, R5] are satisfied.
FR7	The robot shall transport collected objects to the starting point.	Demo	Subsystem Rover base, Manipulator, Navigation, Depth camera	Run mission and verify objects reach starting point without dropping.	Rover collects and returns all objects successfully.	R7 Ensures [FR5, R5] are satisfied.
PR7.1	The robot shall securely hold 3 objects onboard.	Demo + Inspection	Subsystem Rover base, Mechanical chassis	Observe objects stay secured in storage during mission.	Objects never fall from storage.	R7.1 Ensures successful grasping.

FR8	The robot shall sort target objects into bins with matching color.	Demo	Subsystem Rover base, Manipulator, Depth camera	Run sorting operation and verify correct bin placement.	All objects placed in correct bins.	R8 Ensures [FR5, R5] are satisfied.
PR8.1	The robot shall detect red, green and yellow bins.	Inspection	Subsystem Rover base, Depth camera	Place bins and verify detection in robot interface.	All text output matches object class.	R8.1 Ensures [FR5, R5] are satisfied.
FR9	The robot shall operate untethered for at least 20 minutes.	Demo	Subsystem Rover base, Power system	Run full mission and verify >20 min operation.	All components remain functional.	R9 Ensures [PR1.3, R1.3] and power limits are satisfied.
PR9.1	The robot's onboard energy storage shall support all subsystems for 30 min.	Demo	Whole system	Fully charge and run mission; verify subsystems powered for 30 min.	All subsystems remain operational.	R9.1 Confirms subsystems supported with safety margin.
PR9.2	The robot shall not use trailing cables.	Demo	Whole system	Run mission untethered.	Robot completes mission without cables.	R9.2 Confirms FR9 satisfied.
FR10	The robot shall maintain mechanical stability.	Demo + Inspection	Whole system	Run full mission and verify no structural issues.	No structural issues detected.	R10 Ensures stability for safe operation.
PR10.1	All components shall securely remain in place.	Demo	Whole system	Run full mission and verify no shifting.	No shifting of components observed.	R10.1 Ensures task completion.

FR11	The robot shall complete mission within 20 minutes.	Test	Whole system	Run mission 10 times and record completion time.	Robot completes mission within 20 minutes in all trials.	R11 Confirms time requirement.
FR12	The robot shall handle all computational work onboard.	Demo	Whole system	Run mission and observe onboard computation.	All computation handled onboard without external support.	R12 Confirms onboard computation.

*Demonstration

**Rover base – the subsystem that consists of only the main body, the motors, the battery, the intel NUC and the lidar. Other components such as the manipulator are not required for verification.

5.2 Appendix B: Workplace Charter

5.2.1 Purpose

This document is used to define the general style of our teamwork environment and act as a common standard for each team member's behaviours as well as any issues that may arise during or outside the working schedule.

Each team member should read and agree to the rules and guidelines outlines in the document. Regular checks shall be carried out to ensure the solid execution of this charter. Each team member

should also refer to this document first should any concerns occurs.

The document will be updated constantly as the project progresses. This is to ensure all newly occurred constraints or events will be considered to prevent mis-communication.

5.2.2 Principles and Commitments

The workspace should be a motivating, resilient, inclusive and professional environment. This will maximize working efficiency and more importantly the trust and team working spirits between the team members.

Each team member shall commit to:

- Being respectful to everyone in the team.
- Fostering an inclusive workspace.
- Taking ownership of their tasks and meeting deadlines.
- Communicating proactively and asking for help as early as possible.
- Sharing knowledge and ideas to help the team develop as a whole.
- Listening to everyone's ideas .
- Upholding ethical standards and honesty.
- Being open to learning and growth.
- Addressing conflicts constructively.
- Escalating issues through appropriate channels.
- Behaving in a professional manner at all times.

5.2.3 Team Rules

Working hours and Availability:

- Each team member is expected to be present at the labs on Mondays and Wednesdays.
- All deadlines must be respected and delays must be communicated in advance.
- Members shall coordinate meetings based on the availability of the group.

Communication:

- All team communications will primarily be in English.
- Team members shall support each other and share knowledge wherever possible.
- Team members shall avoid unconstructive criticism.
- Team members shall be willing to compromise and problem-solve during discussions.

Safety:

- Team members shall stick to the guidelines stated in the lab risk assessment documents.
- Team members shall ensure their safety and each other's safety at all times.
- Team members shall report any incidents, near misses or unsafe equipment immediately.

Collaboration:

- Members shall take ownership of assigned tasks and deliver on time.
- Team members shall support each other and share knowledge wherever possible.
- Team members shall respond to work-related messages within 24-48 hours.
- All team members shall be encouraged to participate in meetings.

5.2.4 Daily Activity Conflict Avoidance

To avoid escalating situations and getting to the grievance procedure, the team members shall:

- Communicate clearly, honestly and professionally at all times.
- Raise concerns as soon as possible.
- Address any signs of tension within the team as soon as possible.
- Listen to everyone's perspective during group discussions.
- Provide constructive criticism.
- Respectfully escalate issues to module leader when issues cannot be resolved within the team.

5.2.5 Conflict Resolution Guidelines**Definitions:**

Minor: Disagreements and tensions between team members.

Examples: accidental verbal disrespect, slightly overdue of individual workload (1-3 days), arguments over best idea

Moderate: Persisting tensions/disagreements, repeated minor conflicts, hindering of team performance.

Examples: deliberate/repeated verbal disrespect, longer overdue of individual workload (4-7 days), slight physical injury due to mistake

Major: Behaviour that violates the University's policies or puts wellbeing of team members at risk. Repeated moderate issues that are hindering team performance.

Examples: severe verbal insult/slander, long overdue of individual workload (> 7 days), deliberate physical injury

5.2.6 Conflict Resolution Flowchart

Note: The procedure shown in Figure 36 shall be followed whenever any of the above conflicts arises.

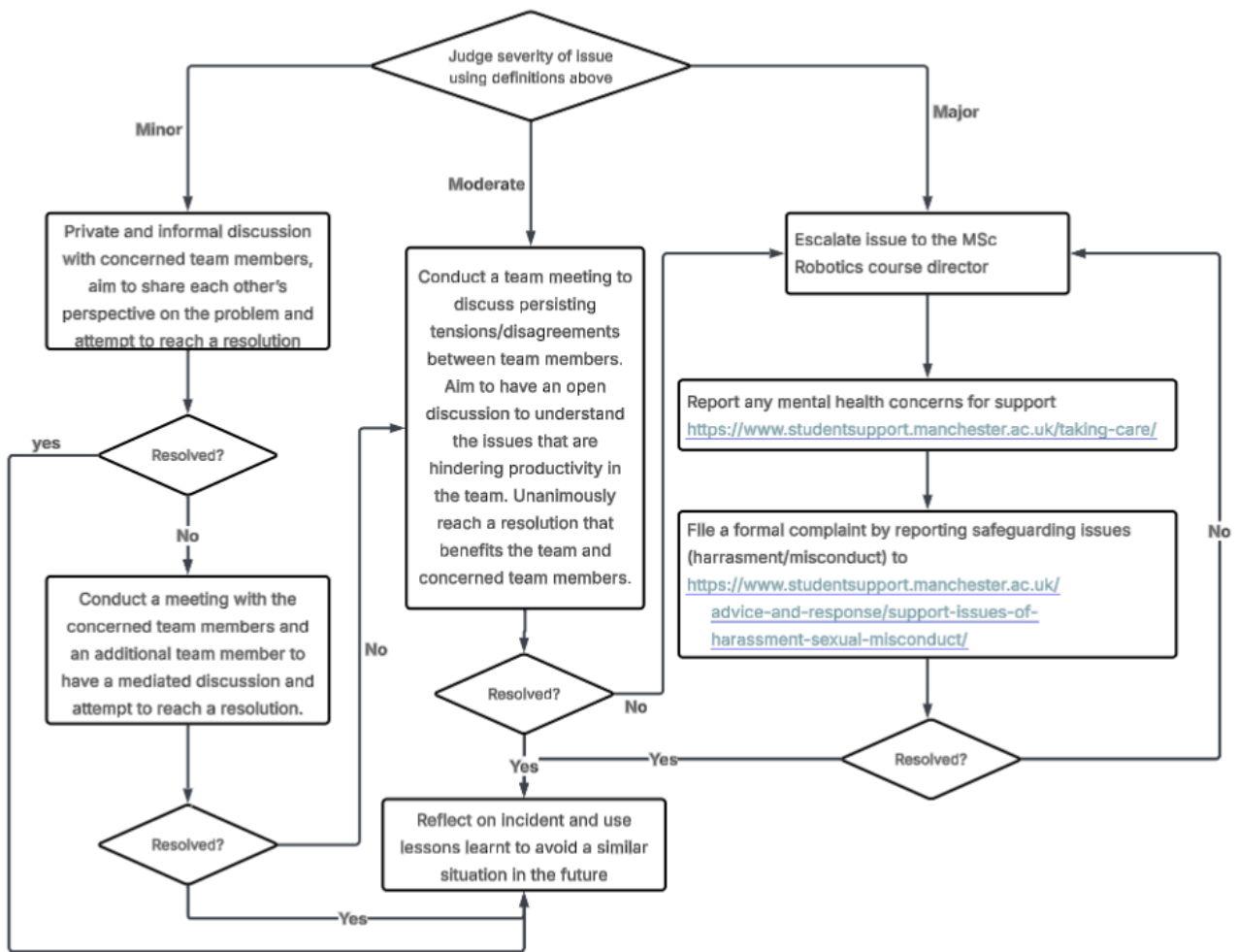


Fig. 36. Conflict Resolution Flowchart